Architecture specification • Revision 2

adatta

Sports Analytics Platform

End-to-End Architecture

Probabilistic football forecasting across every team level: senior and reserve clubs, youth sides, and national teams

Status	Living draft — update as decisions are made or reversed
Revision	2 — restructured by dependency; supersedes the August 2026 edition
Date	November 2026
Scope	Football first, multi-sport-migratable core; all team levels for which data exists
Data source	API-Football (primary); Transfermarkt and Open-Meteo as narrow supplements
Excluded by design	Betting odds as model features (odds appear only downstream, in the Edge/CLV module); in-play recalculation (deferred)

Contents

About this revision 1

Part A — Foundations 2

1 Purpose, scope and principles 2
- 1.1 Purpose and scope . . . 2
- 1.2 Guiding principles . . . 2
- 1.3 Conventions . . . 2

2 System overview 2
- 2.1 Layers . . . 3
- 2.2 Data flow and the feedback loop . . . 3
- 2.3 Deployment shape . . . 3

3 Domain and data model 4
- 3.1 Core entities . . . 4
- 3.2 Team levels, parentage and registrations . . . 4
- 3.3 Temporal data contract . . . 5
- 3.4 Capability fact table . . . 5
- 3.5 Phase 0 subset and end-state schema . . . 5

4 Data sources and ingestion 5
- 4.1 Source map . . . 5
- 4.2 API-Football . . . 6
- 4.3 What the API probably does not provide . . . 6
- 4.4 Feasibility register and capability degradation . . . 6
- 4.5 Data quality and monitoring . . . 9
- 4.6 Entity resolution and retention . . . 10

Part B — Models 11

5 Core team model 11
- 5.1 Global priors (Layer 1) . . . 11
- 5.2 Team strength (Layer 2) . . . 11
- 5.3 Training window and lookback policy . . . 11

6 Context layer 12
- 6.1 Style and tempo (Layer 3a) . . . 12
- 6.2 Venue, weather, referee, congestion, importance (Layer 3b–e) . . . 12

7 Squad and player layer 13
- 7.1 Status feed and availability (Layer 4) . . . 13
- 7.2 Selection, minutes and substitutions . . . 13
- 7.3 Formation prediction and role assignment . . . 13
- 7.4 Player performance (Layer 5) . . . 14
- 7.5 Lineup offset and the player-team feedback loop . . . 14

8 Match synthesis and decomposition 15
- 8.1 Synthesis (Layer 6) . . . 15
- 8.2 Half-split decomposition . . . 15
- 8.3 Goal decomposition and specialty sub-models (Layer 5b) . . . 15
- 8.4 Discipline and cards (Layer 5c) . . . 16
- 8.5 Manual overrides . . . 16

9 Competition formats and knockouts 16

9.1 Format rules . . . 16
9.2 Extra-time sub-model . . . 18
9.3 Penalty shootout . . . 18
9.4 Two-legged ties . . . 18
9.5 Tournament simulation . . . 18

10 Extending across team types **18**
10.1 Global cross-league rating layer . . . 18
10.2 Taxonomy and metadata . . . 18
10.3 Promotion, relegation and transfers . . . 19
10.4 National teams . . . 19
10.5 Youth, reserve and other team levels . . . 20
10.6 Sparse leagues, tiered refits and onboarding . . . 21

Part C — Serving and markets **22**

11 Simulation, markets and projection catalog **22**
11.1 From simulation to markets . . . 22
11.2 Coherence checks . . . 22
11.3 Dependence between sub-models . . . 22
11.4 Projection catalog . . . 23

12 Inference cadence and serving **24**
12.1 Refit tiers . . . 24
12.2 Resimulation timeline . . . 24
12.3 Retention and compute policy . . . 25
12.4 Backup and recovery . . . 25

13 Explainability and confidence **25**
13.1 Decomposition output . . . 25
13.2 Confidence profile . . . 25

Part D — Evaluation and governance **27**

14 Evaluation protocol **27**
14.1 Walk-forward harness . . . 27
14.2 Evaluation span versus training lookback . . . 27
14.3 Development window and locked holdout . . . 27
14.4 Baselines . . . 28
14.5 Inference-quality gates . . . 28
14.6 Champion/challenger promotion . . . 28
14.7 Recalibration . . . 28
14.8 Reproducibility manifest . . . 28

15 Edge/CLV module **28**
15.1 Definitions . . . 28
15.2 Data requirement . . . 29
15.3 Segmentation . . . 29
15.4 Guardrail . . . 29
15.5 Reporting . . . 29

Part E — Product and platform **30**

16 User interface **30**
16.1 Views and build order . . . 30
16.2 Fixture and competition browser (§16.1) . . . 30
16.3 Match projection dashboard (§16.2) . . . 31

16.4 Pre-match override interface (§16.3) . . . 31
16.5 Edge analytics (§16.4) . . . 31
16.6 Model operations (§16.5) . . . 31
16.7 Competition management and coverage map (§16.6) . . . 31
16.8 Settings, notifications and audit (§16.7) . . . 31
16.9 Team profile and player profile (§16.8, §16.9) . . . 31
16.10 Knockout and tournament tracker (§16.10) . . . 32
16.11 Backtest explorer (§16.11) . . . 32
16.12 Override impact review (§16.12) . . . 32
16.13 Fixture congestion calendar (§16.14) . . . 32

17 Technology stack and cost **32**
17.1 Stack . . . 32
17.2 Cost . . . 33
17.3 Budget and quota alerting . . . 33

18 Legal, privacy and continuity **33**
18.1 Regulatory exposure . . . 33
18.2 Data licensing . . . 34
18.3 Privacy . . . 34
18.4 Documentation and bus factor . . . 34
18.5 Incident and postmortem habit . . . 34

Part F — Delivery **35**

19 Roadmap **35**
19.1 Scaling strategy . . . 35
19.2 Getting started: the first weeks . . . 35
19.3 Phases . . . 35
19.4 Timeline . . . 36
19.5 Traceability . . . 37

20 Backlog and deferred items **37**
20.1 Deferred by design . . . 37
20.2 Experiment backlog (run one at a time, after Phase 3) . . . 37
20.3 Not planned . . . 37
20.4 Other deferrals . . . 38

Appendices — Reference material **39**

A Detailed model DAG **39**

B Phase 0 and end-state schema **39**

C Traceability matrix **40**

D API-Football request budget **41**

E Source-feasibility register (template) **42**

F Change history and decision log **42**

G Multi-sport abstraction **43**

H Glossary and notation **44**

List of Figures

1 System overview. The only feedback into the model is the backtest harness; the Edge/CLV module is downstream-only and has no path into Layers 1–7. 3
2 End-state schema, part 1: competitions, rounds, ties and matches. 7
3 End-state schema, part 2: teams, players, registrations, call-ups and lineups. . . . 7
4 End-state schema, part 3: operational and temporal entities (capabilities, status snapshots, overrides, runs, projections, odds). 8
5 Ingestion and data flow. A quota planner schedules source calls; every row keeps its `source_id`; the capability table gates which layers run. 8
6 Knockout tie resolution. Normal-time simulation is unchanged; tie resolution layers on top of it. 17
7 National-team model. Club-level player quality is placed on a common scale and aggregated into a squad offset; the national model has its own strength process, competition-type effects and tournament simulator. 19
8 Team levels. Each level is its own competitor with a prior drawn from its parent and its own dynamics; players bridge levels and put them on one global scale. . . 20
9 Match-day resimulation timeline and persisted snapshots. 24
10 Detailed model DAG. 39

About this revision

This edition keeps the architecture of the August 2026 draft and changes how it is organised and what it assumes. It is arranged by dependency: what the system is and how data arrives (Part A), the models (Part B), serving and markets (Part C), how everything is judged (Part D), the product and platform around it (Part E), and how it gets built (Part F). Earlier drafts placed the capability tiers and multi-league scaling after the roadmap that depends on them.

What changed

- **Data source.** Scraping from Sofascore is replaced by API-Football (§4). Transfermarkt is retained only for market values and contract dates. Covariates that depended on scraped fields (PPDA, directness, distance and sprint data) are re-scoped or moved to the backlog.
- **Team types.** Two new design sections cover national teams (§10.4) and youth and reserve sides (§10.5), with schema, evaluation and legal consequences carried through the document.
- **Evaluation protocol.** Scattered backtesting, gating and versioning rules are consolidated in Part D, with a proposed numeric threshold for the first production-model gate and `team_level` as a standard segment.
- **Open modelling questions resolved.** The half-split versus Dixon-Coles interaction (§8.2) and the way the lineup offset enters the likelihood (§7.5) now have explicit specifications.
- **Corrections folded in.** Inline "Correction" notes from the first draft are rewritten as clean text; the history is kept in Appendix F.
- **Roadmap rebuilt.** Hours-based estimates with buffers, international and levels workstreams, a trimmed first UI release, and snapshot capture moved to the start of the build (§19).
- **Figures redrawn.** The system diagram, schema diagrams, timeline and data-flow diagrams reflect the corrected design; the detailed DAG is in Appendix A.

Conventions. Items marked *[verify]* depend on a provider capability or term that has not been confirmed and must be settled in the data audit (§4.4) before dependent work starts. Projection tiers (1–3) express modelling priority, not product importance. *Competition* is a single entity (the Premier League); a *CompetitionBundle* is a group maintained together (the English top flight and its feeder). The schema is *multi-sport-migratable*, not multi-sport-ready: generalised entities are added when a second sport is onboarded (Appendix G).

Part A — Foundations

What the system is, how data is modelled, and where it comes from. Sections 1–4.

1. Purpose, scope and principles

1.1 Purpose and scope

adatta is a probabilistic forecasting platform for football whose core is deliberately sport-agnostic. It produces posterior-based projections at match, team and player level and derives betting-style markets from simulation. It is built for a small team of analysts first; a commercial audience is a possible later stage, which is why legal and continuity questions are tracked from the start (§18).

In scope. Every team level for which data exists: senior club sides, reserve and B teams, youth sides, senior national teams and youth national teams. Every competition type: domestic leagues and cups, continental competitions, international tournaments and qualifiers, and friendlies. What actually runs for a given competition is decided by its capability profile (§3.4, §4.4), not by a fixed list.

Out of scope by design. Betting odds as model features (odds appear only in the downstream Edge/CLV module, §15); in-play and live recalculation (deferred, §20); taking or placing bets.

1.2 Guiding principles

1. **Vertical slice before breadth.** A validated team-level goals model beats an unfinished platform. Build order is enforced, not aspirational.
2. **Odds-independent.** No odds-derived quantity enters any projection model at any layer. Odds are used only to detect edge, downstream of projections.
3. **Everything is a posterior.** Every projection carries parameter uncertainty through simulation to the market layer. The one exception is the Phase 1a benchmark model (§19.3), which exists only as a comparison floor.
4. **Adjustments never mutate stored inference.** Manual overrides are applied at simulation time, fully logged and fully reversible (§8.5).
5. **Generic core, sport-specific plugins.** Schema and pipeline are sport-agnostic; Dixon-Coles football logic is a plugin (Appendix G).
6. **If it can't be backtested, it doesn't ship.** Every model change is judged by walk-forward proper scoring rules against actual outcomes, never against odds (§14).
7. **Capability-gated.** A layer runs only where the data it needs is verified to exist for that competition and period (§4.4). Missing capability degrades a competition gracefully; it never silently produces a broken projection.
8. **Reproducible.** Every published number is traceable to a manifest of code, configuration, data snapshot, overrides and seed (§14.8).

1.3 Conventions

Projection tiers: Tier 1 (core: dedicated distribution, drives a named market, built early), Tier 2 (secondary: valuable for props, context and explainability), Tier 3 (tertiary: descriptive, derived, or covariate-only). Section references use "§"; phase references refer to §19. *As-of* means "using only what was known at that timestamp", enforced by the temporal contract (§3.3).

2. System overview

2.1 Layers

The model is organised as eight layers (Figure 1). Layers 1–2 estimate league-level priors and team strength; Layer 3 adds match context; Layer 4 predicts who plays and in which role; Layer 5 models individual player rates; Layers 5b/5c decompose goals and model discipline; Layer 6 synthesises a match-level goal process; Layer 7 simulates and derives markets; Layer 8 evaluates everything against realised outcomes. The Edge/CLV module sits outside the model, downstream of Layer 7.

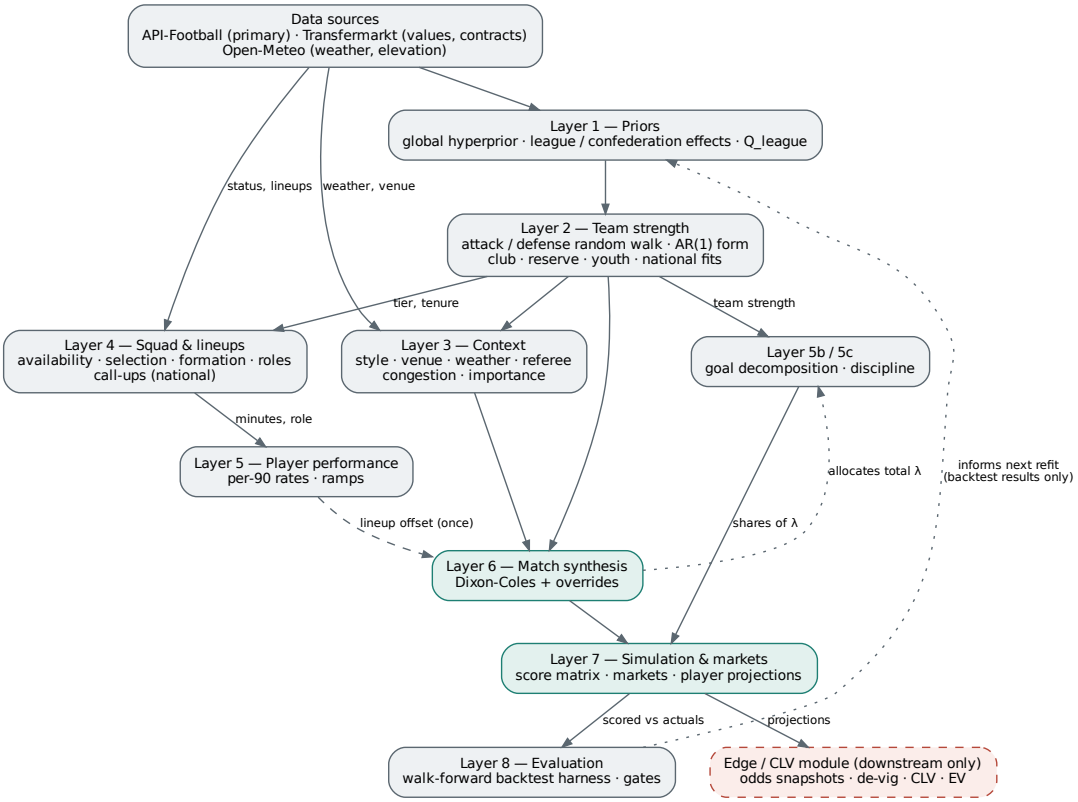


Figure 1: System overview. The only feedback into the model is the backtest harness; the Edge/CLV module is downstream-only and has no path into Layers 1–7.

2.2 Data flow and the feedback loop

Information flows in one direction through the squad layer. Player-level quality (Layer 5), combined with expected minutes (Layer 4), forms a single lineup offset that enters Layer 6 exactly once (§7.5). Player projections are then allocated from the resulting team rate as normalised shares, so team and player projections stay internally consistent by construction. Layers 5b/5c allocate or condition on the synthesised total; they do not add new goal mass on top of it (§8.3).

The only path from outputs back into the model is the walk-forward backtest harness (Layer 8), which informs the next refit through gate decisions and diagnostics. Edge and CLV measurements never feed back into Layers 1–7.

2.3 Deployment shape

Two deployables with different resource profiles: a *compute* service (ingestion, refits, resimulation jobs, scheduled by an orchestrator) and a *serving* service (API reading stored posterior

draws and simulation results, plus the dashboard frontend). State lives in Postgres (relational data, temporal facts, run metadata) and object storage (posterior traces, raw payloads). Details are in §17.

3. Domain and data model

3.1 Core entities

Entity	Purpose	Key fields
Sport, Competition	Sport and its competitions	`archetype`; `competition_type`, `format`, `tier`, `confederation`, `season_calendar`, `age_group`, `overage_rules`
CompetitionBundle	Group of competitions onboarded and maintained together	member competitions
Season, CompetitionRound	Season instances; per-round format rules	`legs`, `extra_time`, `penalties`, `tiebreak_method`, `away_goals_rule` with validity dates
Match	One fixture	`kickoff`, `match_status`, `is_neutral`, `host_flag`, normal-time and extra-time scores, `went_to_extra_time`, `went_to_penalties`
KnockoutTie	Links two legs	`leg1_match_id`, `leg2_match_id`, aggregate scores
Competitor	What a Match references (team, player or pair)	`competitor_type`
Team	Team at a given level	`team_level`, `is_national`, `parent_team_id`, `home_venue_id`
Player, PlayerRegistration	Individual and the teams and levels they are registered with over time	`birth_date`; `valid_from`, `valid_to`
CallUp	National-team call-up list per window	`window`, `status`, `announced_at`
Manager, ManagerTenure	Top-level so tendencies travel between clubs	start and end dates
Venue, Referee	Match context	coordinates, altitude, timezone, surface; fitted strictness
Lineup	Per-match squad slots	`formation`, `slot_code`, grid position, expected minutes
PlayerStatusSnapshot	Daily status (available, injured, suspended, transferred, other)	`as_of`, `expected_return`, `competition_eligible`
CompetitionSourceCapability	Temporal capability facts (§3.4)	competition, season, source, capability, `coverage_ratio`
Override	Structural and parametric overrides (§8.5)	match, entity, field, value, who, when, why
ModelRun, ProjectionRun	Fit metadata and the reproducibility manifest (§14.8)	config hash, commit, trace path; component versions, seeds
MarketProjection, OddsSnapshot	Projections per milestone; paired odds captures (§15)	probability and interval; bookmaker, line, timestamps, de-vig method

3.2 Team levels, parentage and registrations

Every team carries a `team_level` (senior, reserve, U23, U21, U19, U18 and younger) and an `is_national` flag. A youth or reserve side has a `parent_team_id` pointing to its club (or, for national youth sides, to the senior federation team) so a prior can flow from parent to child without merging the two into one strength process (§10.5). Players are registered to teams through the temporal `PlayerRegistration` table, because a player's level changes over time and the same player may appear for several levels in a season. `birth_date` is mandatory for every player: age

drives both development effects and eligibility. Competitions carry `age_group` and `overage_rules` (maximum over-age players, birth-year cutoffs).

3.3 Temporal data contract

Every ingested fact carries the same set of timestamps rather than each data type inventing its own convention: `event_time` (when the real-world event applies), `published_at` (when the source reported it), `observed_at` (when this system first saw it), `ingested_at`, `valid_from`/`valid_to`, and `superseded_at`. This makes the as-of discipline of every backtest enforceable for every field, not only the ones someone remembered to timestamp. Mutable facts (player status, referee appointments, weather values, odds) are stored as snapshots, never overwritten.

3.4 Capability fact table

Data availability is not a fixed property of a competition. It varies by season, source, stage and field, and can change mid-history. Capability is therefore its own temporal fact table:

```
competition_source_capability
  competition_id, season_id, source_id, capability,
  coverage_ratio (0.0-1.0), valid_from, valid_to,
  last_audited_at, status: available | partial | absent | unknown
```

Dashboard badges and layer gating derive booleans for a given season and as-of date from this table. Capabilities are tracked at the granularity the models need: `has_lineups`, `has_player_match_stats`, `has_shot_stats`, `has_xg`, `has_corners_data`, `has_cards_data`, `has_half_split_stats`, `has_physical_load_data`, `has_attendance_data`, `has_event_coordinates`, `has_substitution_timestamps`, `has_referee_history`, plus the player-prop capabilities `supports_lineup_strength_adjustment`, `supports_player_goal_props`, `supports_player_shot_props` and `supports_player_discipline_props`. Regime facts that genuinely change over time (`has_var`, `bench_size_max`, stoppage-time averages) are stored the same way. `last_audited_at` makes the staleness of the audit itself visible.

3.5 Phase 0 subset and end-state schema

Phase 0 builds only what the first phases need: Competition, CompetitionBundle, Season, Team (with `team_level`, `is_national`, `parent_team_id`), Match, Player, PlayerRegistration, PlayerStatusSnapshot, the capability table, the temporal-contract columns, the override tables, and the model-run and projection-manifest tables. The generalised multi-sport entities (Competitor polymorphism, Period, generalised MatchEvent) are deliberately not built until a second sport is onboarded; Appendix G gives the migration path. Figures 2–4 show the end-state schema in three parts.

4. Data sources and ingestion

4.1 Source map

Source	Role	Cadence	Notes
API-Football	Primary: fixtures, results, lineups, events, match and player statistics, injuries, transfers, squads, pre-match odds	Backfill once; daily; match-day polling	Licensed API with a request quota; covers many competition types and levels, depth varies
Transfermarkt	Narrow: market values (cold-start prior), contract dates	Weekly or on demand	Prior and covariate candidate only; not on the critical path for match-day status
Open-Meteo	Weather forecast (about 16 days), climatology, venue elevation	On forecast refresh; elevation cached once	No key, no cost

Source	Role	Cadence	Notes
Odds	Pre-match snapshots for CLV (§15); historical closing odds if needed	Forward capture from day one	Historical depth in the API must be confirmed; a second source may still be needed for history *[verify]*

4.2 API-Football

Plan and quota. The Pro plan provides 7,500 requests per day with a rate limit around 300 per minute; every plan includes all endpoints and competitions, and calls stop at the daily limit rather than billing overage. The next tier is an order of magnitude larger, so the upgrade path is a step, not a slope. Current terms and prices must be confirmed on the provider's pricing page before budgeting *[verify]*.

Endpoints used (confirmed field by field in the audit, §4.4):

Endpoint family	Used for	Notes
Leagues (per-season coverage object)	Competition discovery; input to the capability audit	Reports which feature groups exist per league-season *[verify]*
Fixtures (by league, season, date, or id batch)	Calendar, results, per-match detail	Batched id lookups may return events, lineups, statistics and player stats in one call *[verify]*
Fixture events	Goals, cards, substitutions with minutes	Source of substitution timing and half-split data
Fixture lineups	Formation string, starters, bench, grid positions	Slot codes derived from formation plus grid (§7.3)
Fixture and player statistics	Shots, possession, corners, fouls; xG where covered; per-player minutes and counts	xG coverage varies by league and season *[verify]*
Injuries, sidelined, transfers, squads	Daily status feed, squad lists	Expected-return detail may be thinner than the previous source *[verify]*
Odds	Pre-match snapshots	Retention window limits history *[verify]*

Quota planner. Cadence is bounded by requests as well as compute. The planner keeps a daily budget with these rules: (1) allocate by refit tier (§12.1), so high-interest leagues get match-day polling and long-tail leagues are updated by date queries; (2) use batched lookups wherever available; (3) reserve a fixed share for match-day lineup and status polling, which is time-critical; (4) record actual usage from response headers and alert on deviation (§17.3); (5) trigger a plan upgrade when sustained planned usage exceeds about 70% of quota, or for a one-off backfill that would otherwise block the pipeline for days. Appendix D gives order-of-magnitude budgets.

4.3 What the API probably does not provide

Several fields assumed by the first draft are probably unavailable *[verify]*: PPDA and directness, distance and sprint (physical load) data, event coordinates, market values, contract end dates, and reliable expected-return dates for injuries. The consequences are explicit: style covariates shrink to the match statistics the API does expose (§6.1); the workload-based injury-risk signal is dropped unless `has_physical_load_data` is verified (§7.2, §20); and Transfermarkt remains for market values and contracts. A deterministic injury override is only as strong as the return-date information available (§7.1).

4.4 Feasibility register and capability degradation

Before any covariate that assumes a field is committed, the **source-feasibility register** records, per field: source and endpoint, historical availability, competitions covered, coverage percentage, update cadence, as-of reconstructability, licensing status, fallback, and the phase that

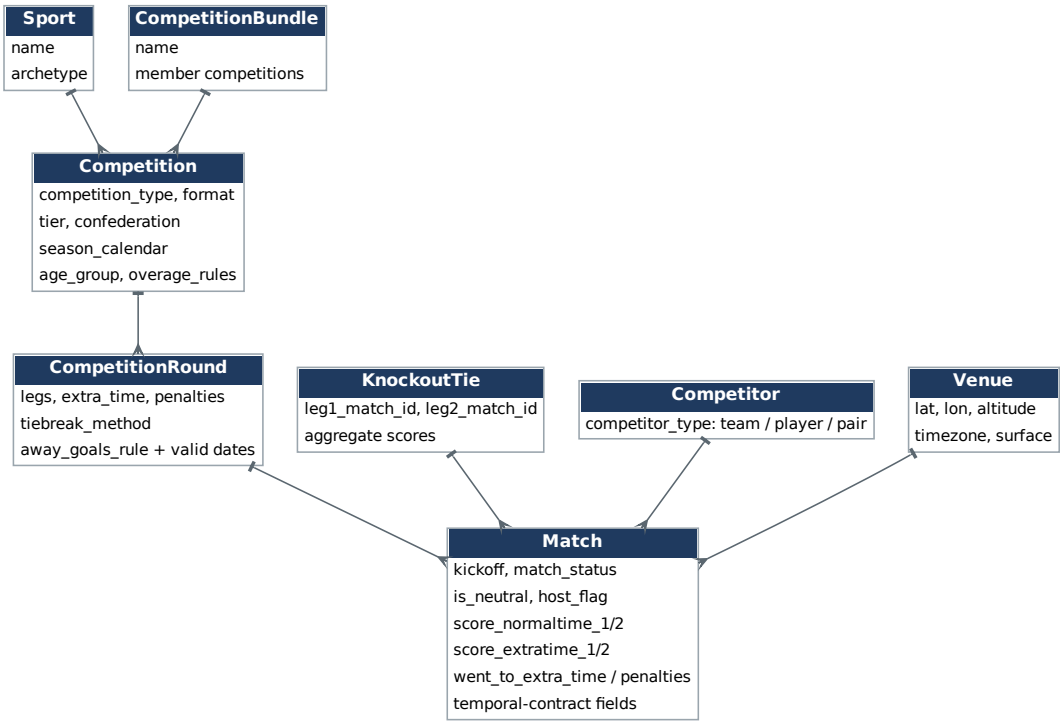


Figure 2: End-state schema, part 1: competitions, rounds, ties and matches.

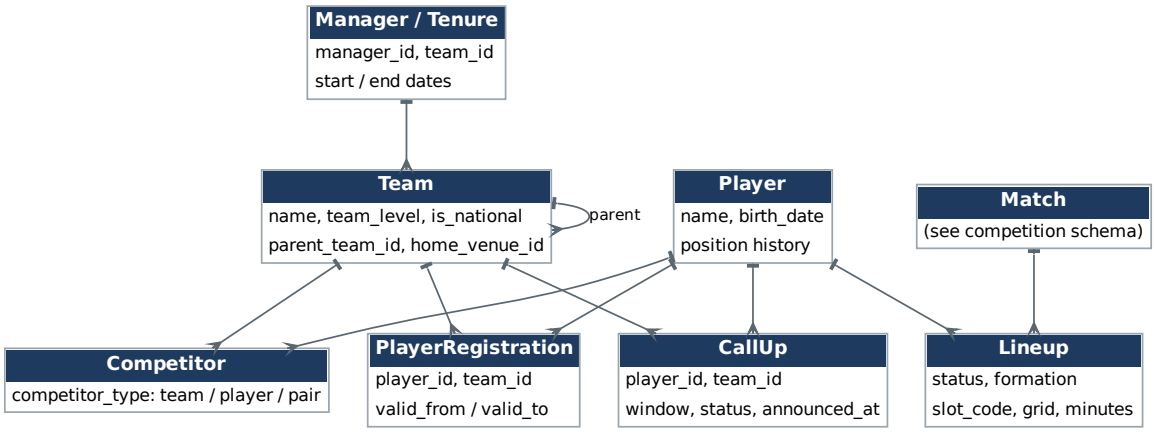


Figure 3: End-state schema, part 2: teams, players, registrations, call-ups and lineups.

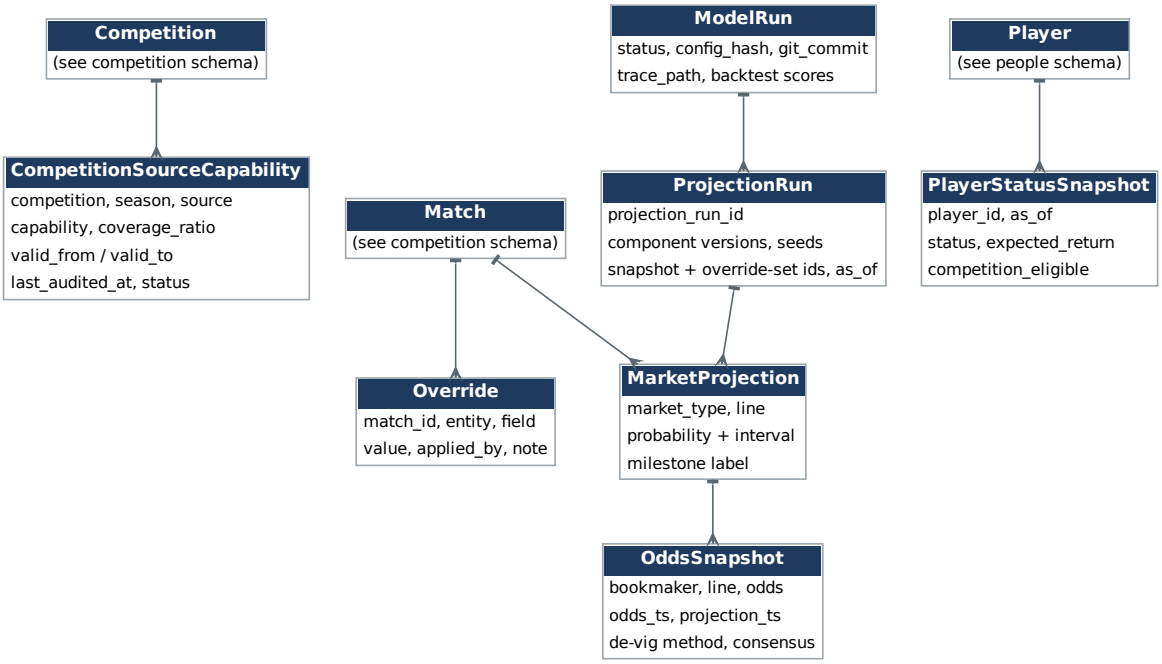


Figure 4: End-state schema, part 3: operational and temporal entities (capabilities, status snapshots, overrides, runs, projections, odds).

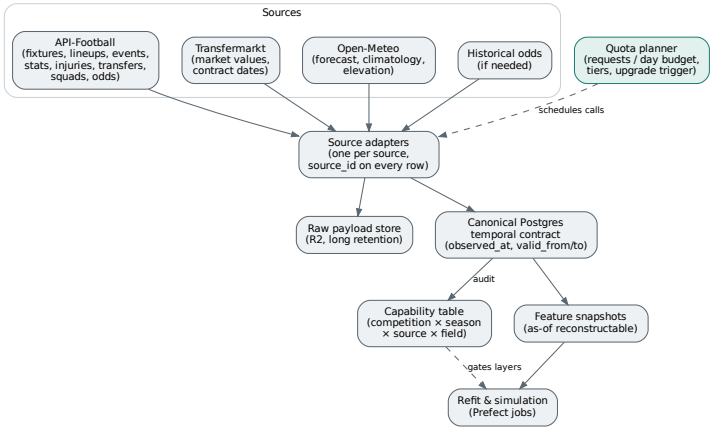


Figure 5: Ingestion and data flow. A quota planner schedules source calls; every row keeps its `source_id`; the capability table gates which layers run.

needs it (template in Appendix E). The API's per-league coverage flags seed the capability table; the audit then verifies them against real payloads, because declared and actual coverage differ.

The baseline is Dixon-Coles team strength from goals alone (Layers 1, 2, 6, 7). It needs only teams, date and final score, so every onboarded competition gets at least this level. Missing capabilities switch layers off as follows:

Missing capability	Effect
`has_xg` / `has_shot_stats`	xG auxiliary signal in Layer 2 skipped; team strength fits on goals alone
Shot, possession and related statistics	Layer 3 style covariates absent from the design matrix (not forced to zero)
`has_lineups`	Layer 4 cannot run: no squad lists or player-level projections; match-level projections still run
Lineups and lineup-strength support, but no player-prop capabilities	Squad list, formation and selection run; a coarse lineup-strength adjustment uses the market-value prior; individual player props are suppressed rather than offered on a permanent prior
`has_corners_data`	Set-piece sub-model and corner markets not offered
`has_cards_data`	Discipline sub-model and card/foul markets not offered
`has_half_split_stats`	1H/2H decomposition does not run; full-match projections only
`has_physical_load_data`	Workload-trend risk signal absent
`has_attendance_data`	Dynamic home-advantage modifier absent

The dashboard and projection catalog surface only the markets a competition's capability profile supports. The flags also reduce compute: a competition without lineups never runs Layers 4–5 or the lineup-triggered resimulations.

4.5 Data quality and monitoring

- **Match status.** An explicit `match_status` (completed, abandoned, voided, walkover) filters non-completed matches before they reach Layers 1–6.
- **Normal time versus extra time and penalties.** Scores are stored as `score_normaltime`, `score_extratime` (extra-time goals only, not cumulative) and `penalty_shootout_result`, with `went_to_extra_time` and `went_to_penalties` flags. Shootout goals are never goals in any rate model. Layers 2/5b/5c train on normal time only, because matches reaching extra time are a selected sample; extra-time goals feed only the dedicated sub-model (§9.2). This separation is built into ingestion from the first line of code.
- **Friendlies.** Senior club friendlies are a different generating process (heavy rotation, experimental lineups, low intensity) and are excluded from core club training by default. Their narrow legitimate uses are lower-weight bridging matches, a weak pre-season cold-start nudge, and fitness signals. National-team and youth friendlies are a large share of their data and are handled with competition-type effects instead (§10.4, §10.5).
- **Schema drift.** Response-shape checks on API payloads, with the adapter layer isolating the rest of the pipeline from changes.
- **Freshness.** Alerts when no new matches arrive for an active league within an expected window, and when next week's fixture calendar is not loaded.
- **Score sanity checks.** Run continuously rather than as a one-off backfill fix.
- **Prediction drift.** Output-side alerts when a league's average projected goals or another aggregate market statistic shifts sharply (for example 20% or more week over week) without a cause; this usually signals a silent pipeline bug.
- **Source resilience.** The pipeline depends on one primary provider. The adapter layer and `source_id` on every row keep a second provider swappable; a fallback is chosen per critical feed (fixtures, lineups, status).

4.6 Entity resolution and retention

API-Football identifiers are the canonical keys for teams, players, fixtures and competitions, which removes most fuzzy matching. Cross-source linking (API to Transfermarkt, and any odds source) uses normalised names plus fuzzy matching with a manual review queue for low-confidence matches, built once rather than patched per league.

Raw payloads are retained much longer than in the first draft. Re-fetching costs quota, so raw responses are kept in object storage for at least a year in compressed form, while parsed structured data in Postgres is permanent. This also protects against a provider changing or withdrawing historical data.

Part B — Models

Priors and team strength, context, squads and players, match synthesis, knockouts, and extension across team types. Sections 5–10.

5. Core team model

5.1 Global priors (Layer 1)

- League baseline: $\mu_{league} \sim \text{Normal}(\log 1.35,\ 0.3)$, a weak prior on the log goal rate.
- Spread hyperpriors: $\sigma_{attack},\ \sigma_{defense} \sim \text{HalfNormal}(0.5)$; $\sigma_{home_adv} \sim \text{HalfNormal}(0.3)$.
- μ_{league} and the σ terms are drawn from a global hyperprior (§10.1), not fitted per league in isolation. This pools variance estimates and lets a thin league shrink toward a sensible global range. It does **not** by itself make team strengths comparable across leagues: that identification is done by the league-quality parameter Q_{league}, bridging matches and external cold-start priors (§10.1), none of which exist for every league pair. Any cross-league prediction without real bridging support for at least one side is prior-dominated on that dimension and must say so through the confidence profile (§13.2).

5.2 Team strength (Layer 2)

- **Random walk.** Attack and defense follow non-centred Gaussian random walks: $a_t = a_{t-1} + \sigma_{walk} z_t$, $z_t \sim \text{Normal}(0,1)$, $\sigma_{walk} \sim \text{HalfNormal}(0.15)$.
- **Short-term form.** An AR(1) layer, $m_t = \phi\, m_{t-1} + \epsilon_t$ with $\phi \in (0,1)$, captures form separately from the slow strength walk.
- **Home advantage.** Hierarchical: $h_{team} \sim \text{Normal}(h_{league}, \sigma_{home_adv})$. An explicit `is_neutral` flag zeroes or sharply reduces it for neutral-venue fixtures, otherwise the model misattributes strength to whichever side is listed as home. Where `has_attendance_data` holds, an expected-attendance-to-capacity ratio acts as a multiplicative modifier, since part of home advantage is crowd-driven; expected attendance can itself be a simple historical model by fixture type, opponent and weekday.
- **xG as an auxiliary signal.** Where `has_xg` holds, fit attack and defense jointly on goals and xG (a joint likelihood, or a two-stage model where xG-implied strength informs the goal-strength prior). Goals are a small-sample, high-variance signal; xG stabilises much faster, which improves early-season and cold-start estimates. This is a core methodology decision, not a Layer 3 covariate, and it is scheduled in Phase 1b with its data-availability gate (§19.3).
- **Regime changes.** A `manager_change` flag temporarily widens σ_{walk} for a defined post-change window, so the model adapts quickly to a discontinuity instead of staying anchored to pre-change form. The same mechanism, at lower priority, applies to major transfer-window overhauls.

5.3 Training window and lookback policy

Lookback is answered per sub-model, not as one global window.

- **Core team strength:** no hard cutoff. State is carried forward indefinitely and σ_{walk} discounts old matches naturally. At season transitions the prior season's final state is carried forward with inflated variance (a soft reset), not truncated. Retaining roughly 3–5 seasons (150–200 matches per team) is a practical data-hygiene default, to be validated by backtest, not a statistical requirement.
- **Everything else uses an explicit bounded window**, because these quantities are not discounted automatically:

Sub-model	Recommended window	Why bounded
League hyperparameters	~3–5 seasons, pooled	Pooling across teams reduces the need for long single-team history
Style and tempo covariates	~1–2 seasons, rolling	League-wide tactical trends shift over a few years
Referee strictness, discipline	~1–2 seasons, rolling	Rule-interpretation changes (VAR, handball) date older data
Manager formation tendencies	Manager's career tenure	A manager-level profile; resets with the manager
Player rate statistics	~2–3 seasons, recency-weighted	Ramp covariates handle the rest
Lineup cohesion	Current season, occasionally prior	Chemistry resets each transfer window
Style-matchup interaction	Whatever exists, heavily shrunk	Shrinkage does the discounting

- **Backtest consistency.** Whatever window policy production uses must be replicated exactly in every historical refit of the backtest. A backtest that sees more history than production ever would is a form of leakage and overstates performance (§14.2).
- **Different time scales by team type.** National teams play about 8–15 matches a year and youth squads turn over every season, so their walks run on calendar time with different resets (§10.4, §10.5).

6. Context layer

6.1 Style and tempo (Layer 3a)

- Possession, shot volume and any other style statistics the API exposes enter as covariates on log λ with Normal$(0, 0.2)$ priors, or a horseshoe prior if only a few are expected to matter. PPDA and directness are included only if the audit (§4.4) verifies them; otherwise they are absent.
- These covariates modulate variance as well as the mean: two high-pressing teams make a higher-variance goal environment even at unchanged λ.

6.2 Venue, weather, referee, congestion, importance (Layer 3b–e)

- **Venue.** Fixed effects for altitude, surface and pitch-dimension buckets, mostly non-zero only in continental and international play. Use the altitude *differential* (away team's home altitude versus this venue) rather than raw elevation. Altitude is looked up once per venue and cached.
- **Weather.** Temperature, wind and precipitation on log λ and on style and variance. Reliable forecasts exist only about 16 days out; before that use climatology for the venue and date, then resimulate when the forecast arrives (§12.2).
- **Travel burden.** Great-circle distance and timezone shift from the team's home venue to the match venue (not previous venue to next), since squads reconvene at their home base between fixtures. Zero for home fixtures. Consecutive away trips without a return home get a specific flag when they matter.
- **Referee.** A hierarchical strictness parameter, primarily for cards and fouls (§8.4) and secondarily for pace. Before assignment is confirmed, draw strictness from the pool of referees typically assigned to that competition and tier; on confirmation, resimulate with the actual referee.
- **Congestion, backward and forward.** Days since the last match as a decay covariate on attack and on injury risk, combined with travel burden. A forward-looking `days_to_next_significant_match` (weighted by importance) enters the rotation-policy prior in selection (§7.2), because rotation depends on what is coming as much as on what has been played.

- **Importance.** Categorical (dead rubber, must-win, final), on λ and on the rotation prior.
- **VAR and stoppage time.** `has_var` and per-half average added time are competition-level temporal facts (§3.4). Tracking them separately per half matters because first-half added time is systematically shorter; they feed expected minutes and, weakly, each half's λ.
- **Kickoff time of day.** A minor covariate candidate under a horseshoe prior; the backtest decides whether it earns a non-zero coefficient.

7. Squad and player layer

7.1 Status feed and availability (Layer 4)

- **Source.** The API's injuries, sidelined, transfers and squads endpoints are refreshed daily, with match-day polling for tier-one leagues (§4.2). Status categories per player and as-of date: available, injured, suspended, transferred out, on loan, other unavailable.
- **Deterministic override.** Where a status is a hard fact (suspension, transfer) it overrides the statistical availability model entirely. An injury with a clear expected return is close to a hard fact; an injury *without* return information is handed to the statistical model. Because the API may supply thinner return-date detail than the previous source *[verify]*, the share of injuries that fall into the statistical bucket is expected to be larger and must be measured in the audit.
- **Statistical availability.** $P(\text{available}_p)$ is a hierarchical logistic by player with recency-weighted injury and suspension history, used only for the residual uncertainty.
- **Snapshots.** Daily status snapshots, never a single mutable field, so a backtest for a January match cannot know about a February injury report. These snapshots cannot be reconstructed later, so capture starts in Phase 0 (§19.2).
- **Competition eligibility.** `competition_eligible` is a best-effort field defaulting to true, corrected through the override interface (§16.3), because registration lists are not reliably available in structured form.

7.2 Selection, minutes and substitutions

- **Selection.** $P(\text{start}_p \mid \text{available}_p)$ is hierarchical multinomial/logistic, conditioned on formation (§7.3), a rotation-policy prior informed by importance and congestion, and historical selection frequency by position.
- **Squad list per match.** The eligible pool is the squad minus unavailable players. From it the models produce 11 starters, N bench players where N is a per-competition setting (`bench_size_max`, not a hard-coded number), and every remaining eligible player as an implicit healthy scratch, a category distinct from *out*.
- **Output is a distribution.** Per-player start probability and an expected-minutes distribution (Beta-scaled around typical minutes per role, zero-inflated for did-not-feature), scaled by the competition's per-half added time. A starter's first-half minutes are 45 minutes plus the average first-half added time in expectation, with a small first-half dropout probability.
- **Substitutions.** Number and timing per team are available from fixture events. They refine expected minutes (a striker substituted at about 65' differs from one who plays 90') and inform variance in the style model. They are used as historical, pre-match features only, since substitutions are themselves a response to game state; in-play use is out of scope (§20).
- **Workload-trend injury risk.** An acute-to-chronic workload ratio signal needs distance and sprint data. If `has_physical_load_data` is not verified it is dropped; it is a backlog item contingent on a data source (§20).

7.3 Formation prediction and role assignment

- **Formation.** A hierarchical multinomial over a defined formation vocabulary, $P(\text{formation} \mid \text{team}, \text{opponent tier}, \text{manager}, \text{context})$, from the team's recency-weighted tendency, an opponent-tier effect (tier from existing strength estimates, or from Q_{league} across leagues),

and a manager-level tendency pooled across the manager's career with shrinkage for short tenures. The output is a distribution, propagated into selection and role assignment rather than collapsed.

- **Slot codes.** The API gives a formation string plus a grid position for each starter, not named slot codes. Slot codes (GK, RB, RWB, RCB, CB, LCB, LB, LWB, CDM, CM, CAM, RW, LW, ST and so on) are derived deterministically from formation plus grid. Finer tactical labels (false nine, inverted winger) are not available as structured data and are not assumed.
- **Role assignment.** Given a sampled formation and likely XI, each player is assigned the slot the formation requires, using the distribution of codes the player has actually occupied. Layer 5 rates condition on the assigned code, pooled code → position family → player, so a right-back deployed as a right wing-back shifts toward more crossing and expected assists. The assignment propagates to the lineup offset and to player projections.

7.4 Player performance (Layer 5)

- **Rates.** Per-90 rates (goals, assists, shots, key passes, tackles, cards) as Poisson or negative binomial, tested for dispersion by position and statistic rather than assuming NB everywhere. Hierarchical pooling player → position → league.
- **Cold start.** Transfermarkt market value, segmented by position, is transformed into a position-specific rate prior for new or sparse-history players, fitted on players with real history and faded out as minutes accumulate. It is a prior only, never a live feature, because market value is an aggregated and sometimes lagging judgement that does not meet the backtesting bar.
- **Ramps.** Injury-return (`matches_since_return`) and new-signing integration (`matches_since_signing`) covariates stop hierarchical pooling from misreading a temporary dip as true rate.
- **Own goals.** A player's projected goals are their own scored goals only; opponents' own goals are never credited to a scorer and own goals scored by the player never count in the tally.
- **Contract situation.** A low-confidence candidate covariate under strong shrinkage, tested and expected to shrink toward zero (backlog, §20).

7.5 Lineup offset and the player-team feedback loop

The mechanism runs in one direction and allocates player quality exactly once.

1. Latent player-impact parameters (rate quality, §7.4) combined with expected minutes (§7.2) determine a lineup offset Δ_{lineup}.
2. That offset enters team λ **once**, as a single additive log-space term in Layer 6 (§8.1).
3. The resulting non-own-goal team rate is allocated among players as normalised shares: $E[\text{goals}_i] = \lambda_{team,\,non\text{-}OG}\, s_i$ with $\sum_i s_i = 1$, where s_i depends on player quality, role and expected minutes. Player expectations therefore sum to the team component by construction.
4. Own goals stay outside this allocation: they are credited to the benefiting team's total but never distributed among its players.

How the offset enters the fit. Two designs are possible, and the choice matters because injury-driven variance otherwise leaks into the strength walk.

- *Serving-only offset.* The team-strength model is fitted without lineup information and the offset is applied only at simulation time. Simple, but the random walk absorbs lineup-driven performance as true strength change, inflating σ_{walk}, and the serving-time offset then partly double-counts.
- *Offset inside the likelihood* (**adopted**). Historical match lineups, with their realised minutes and player quality, produce a historical offset $\Delta_{lineup,t}$ that enters the training likelihood, so the walk tracks *baseline* strength with a typical lineup. At serving time the offset is the expected offset for the projected XI, measured against that baseline. This requires `has_lineups` history for the training period; for competitions without it the offset is absent and the walk absorbs lineup effects, which is flagged in the confidence profile.

Phase 3 therefore re-fits Layer 2 with the offset in the likelihood and must show on the locked holdout that this beats the serving-only variant before the feedback loop is wired into production (§19.3).

8. Match synthesis and decomposition

8.1 Synthesis (Layer 6)

$$\begin{aligned}\log \lambda_{home} = \mu_{league} + a_{home} - d_{away} + h_{home} \\ + \text{style} + \text{venue} + \text{ref} + \text{cong} + \text{imp} \\ + \Delta_{lineup,home} + \text{override}_{home}\end{aligned}$$

The away rate mirrors this without the home-advantage term. The Dixon-Coles low-score correction ρ has its own weak prior, possibly hierarchical by league. Models are fitted by NUTS (PyMC/ArviZ, with NumPyro when a league's refit exceeds its time budget, §12.3). Match-day serving is posterior-predictive simulation off the stored draws; live NUTS is never run per request.

Because the synthesis is a sum of log-space terms, the decomposition is a defined output (§13.1), not an implementation detail.

8.2 Half-split decomposition

Every projection can be split into first and second half where `has_half_split_stats` holds. It is not two independent models: goals are not evenly distributed across halves, and independent half models would both halve the effective sample for the hardest estimation problem and fail to guarantee that halves sum to the full match.

- **Mechanism.** A split-share parameter $\text{share}_{1H} \sim$ Beta, hierarchically pooled by team and league, gives $\lambda_{1H} = \lambda\,\text{share}_{1H}$ and $\lambda_{2H} = \lambda(1-\text{share}_{1H})$. By Poisson thinning, if goals are Poisson(λ) and each falls in the first half with probability p, the halves are Poisson(λp) and Poisson($\lambda(1-p)$) and sum back to the original process. Full-match data fits team strength; the observed conditional split fits the share.
- **Each sub-model has its own share.** Corners, cards, penalties and own goals each get a `share_1H`; cards skew later than goals, for instance. Player-level splits respect the expected-minutes machinery, since a player substituted at 60' accrues second-half exposure only to that minute.
- **Refinement once validated.** Let the share depend on match covariates (congestion, importance, formation) through a logistic or beta regression, backtest-gated like any covariate.
- **Resolution of the Dixon-Coles interaction.** The full-time score matrix carries the low-score dependence ρ; two independently drawn half matrices would not reproduce it, so "1H + 2H reconciles with FT" would fail by construction. The rule is: **the full-time matrix is primary.** Half-time and half-based markets are derived as conditional splits *of simulated full-time outcomes*: within each simulation draw, the N full-time goals of each team are allocated to halves by the share (binomial thinning given N), so half scores always sum exactly to the full-time score and the full-time matrix retains ρ. This introduces no independent half-time dependence beyond what thinning implies, which is stated as a known simplification (no causal effect of the half-time score on second-half tactics, consistent with in-play being out of scope).
- **Coherence checks** (§11.2): within-draw half scores sum to full time; HT/FT combined probabilities sum to one across the nine combinations and marginalise to the 1X2 and half-time 1X2.

8.3 Goal decomposition and specialty sub-models (Layer 5b)

Layer 2 is trained on total normal-time goals. Adding penalty, set-piece and own-goal rates on top of that total would double-count goals already inside it. The design is **total-first**: λ_{total} is

allocated, not added to.

- share_{pen}, $\text{share}_{setpiece}$, share_{og} are fitted proportions of λ_{total} (small hierarchical rates relative to the whole), with $\text{share}_{open} = 1 - \text{share}_{pen} - \text{share}_{setpiece} - \text{share}_{og}$ absorbing the remainder.
- **Penalties.** Awarded and converted are separate small models (conversion roughly 76–78% league-wide, pooled by taker where data allows). Output is a share of the total and a "will there be a penalty" market. VAR-enabled competitions show different award and overturn rates.
- **Set pieces.** A lightweight dedicated rate model on corners won and conceded drives corner markets directly; the set-piece goal share respects the total-first constraint.
- **Own goals.** About 2–3% of goals. The generating process belongs to the conceding team: the own-goal share is carved from the conceding team's allocated total and credited to the opponent, driven by pressure from the benefiting side.
- These sub-models feed Layer 7 markets directly from their share parameters; they do not re-enter Layer 6. Moving to a component-generating design (Layer 2 models open play only) is a legitimate later migration once event-level goal-type data is reliable, not the default.

8.4 Discipline and cards (Layer 5c)

- A dedicated sub-model: $\lambda_{cards,team} = f(\text{referee strictness}, \text{team foul tendency}, \text{opponent foul tendency}$ with team foul tendency pooled team → league.
- Referee strictness is the primary driver, a hierarchical parameter with its own historical cards per match.
- Player cards condition on the team rate plus individual foul history and position (defensive roles carry higher baseline rates). Yellow and red cards are separate processes with heavy shrinkage, since red cards are rare and partly driven by distinct events.
- Output feeds markets directly and Layer 6 only as minor context.

8.5 Manual overrides

Two structurally different mechanisms, deliberately not collapsed.

- **Parametric scale factors.** Numeric adjustments stored as (match, team or player, attribute, additive or multiplicative, value, applied_at, applied_by, note). Applied only as an additional log-space term at simulation time; posterior draws are never modified, so with-versus-without comparisons are always reconstructable.
- **Structural overrides.** Direct replacement of any normally scraped or inferred field for an upcoming match: availability status, predicted formation, a lineup slot, referee, weather values, venue details, `competition_eligible`, call-up status. A general-purpose table (match, entity type, entity, field, value, who, when, why), read at simulation time with the precedence *explicit override > inferred value*, never mutating the underlying data.
- **Shared properties:** applied at simulation time only; fully logged; fully reversible; never baked into stored posteriors or source data. Each override triggers an immediate resimulation of that match.

9. Competition formats and knockouts

Normal-time projections remain the primary output for every match. This part adds the layer needed where a winner must be determined (Figure 6).

9.1 Format rules

Format belongs on a `CompetitionRound`, not on the competition, since one competition mixes formats. Fields: `legs` (1 or 2), `extra_time`, `penalties`, a `tiebreak_method` enum (ET plus penalties, penalties only, replay), and `away_goals_rule` with its own validity dates (UEFA abolished it for club competitions from 2021-22; history must match the rule in force at the time, for both training data and backtest integrity).

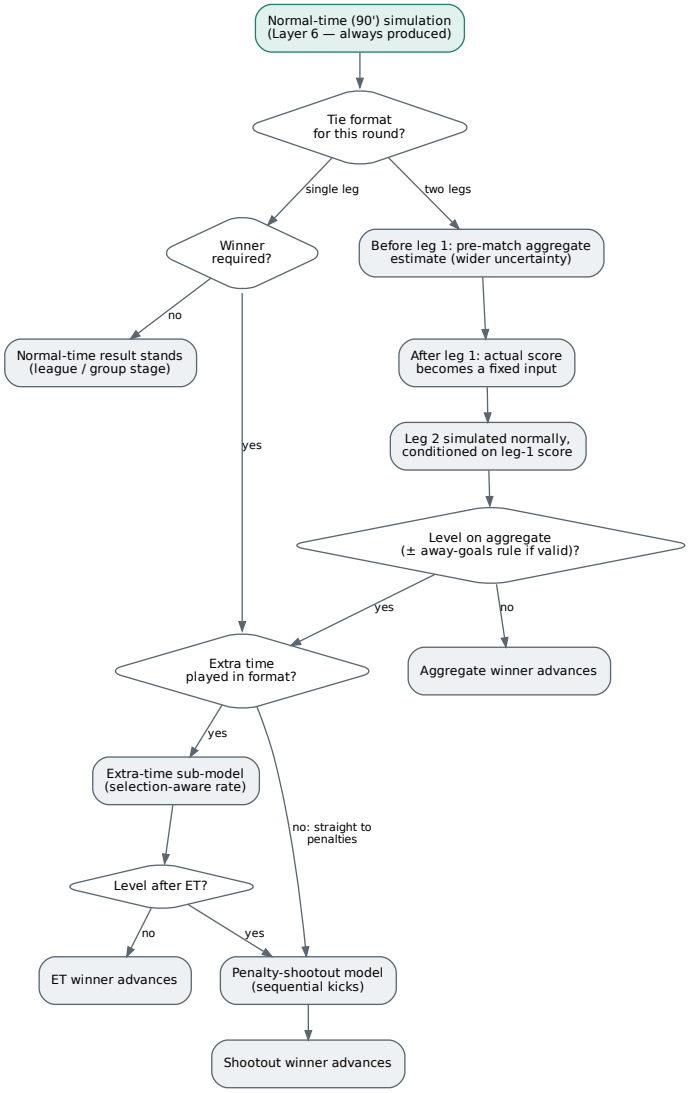


Figure 6: Knockout tie resolution. Normal-time simulation is unchanged; tie resolution layers on top of it.

9.2 Extra-time sub-model

Applies only where `went_to_extra_time` is true. Matches reaching extra time are a selected sample of teams that failed to separate in regulation, so λ_{ET} is its own small hierarchical rate (per team, conditioned on being level), not $\lambda_{total} \times 30/90$. The direction of the selection effect is left to the backtest. Samples are small, so it leans on pooling and wide intervals.

9.3 Penalty shootout

A separate process from open play: taker and goalkeeper conversion and save rates, pooled player → team → league from in-shootout and in-match penalties. Simulated kick by kick, five kicks each then sudden death, with $P(\text{scored}) = f(\text{taker}, \text{goalkeeper})$ rather than a coin flip. A small home-side edge is a backtest-gated covariate. Never blended with open-play training data.

9.4 Two-legged ties

A `KnockoutTie` links two matches; the aggregate is a genuine cross-match dependency.

- Before leg 1: a pre-match aggregate estimate from both legs' projections, with wider uncertainty.
- After leg 1: leg 2 conditions on leg 1's *actual* score, with the away-goals rule where valid.
- Advancement: $P(\text{adv}) = P(\text{win on agg.}) + P(\text{level})\,[P(\text{win ET}) + P(\text{level ET})\,P(\text{win pens})]$. A single-leg match uses the same form with the 90-minute result. Where the format has no extra time, the bracket goes straight to penalties.

9.5 Tournament simulation

For group-then-knockout events and national-team tournaments (§10.4), a Monte Carlo over the whole bracket produces group-qualification, round-reach and title probabilities, reusing the tie-resolution logic above plus format-specific rules: group tiebreakers in order, best-third-place qualification in expanded formats, and seeding and draw constraints. Each simulated tournament uses one posterior draw per team, so correlated uncertainty is preserved across the bracket.

10. Extending across team types

10.1 Global cross-league rating layer

Teams from different leagues meet in continental competitions, so per-league models need a shared scale. Above the league hyperpriors sits a global layer: $\mu_{league} \sim \text{Normal}(\mu_{global}, \sigma_{global})$ and a league-quality parameter Q_{league} that rescales team strength onto a common axis.

- **Estimation.** Q_{league} is estimated from *bridging matches*: continental fixtures, international tournaments and qualifiers, and cross-league friendlies, weighted by reliability (competitive over friendly). It is refit when bridging matches accumulate, separately from domestic refits.
- **Cold start.** Most leagues have no bridging data for years. A third-party league-reputation scale supplies an initial Q_{league} through a transform fitted on leagues where a bridged estimate exists, and is progressively overridden as real matches accumulate. It is a prior only, never a feature in log λ; large sustained disagreement with the fitted value is a signal to investigate. Its licensing is part of the data review (§18.2).
- **Priority.** Continental and international competitions are scraped and modelled early because they carry disproportionate value for calibrating the scale.

10.2 Taxonomy and metadata

Each `Competition` carries `competition_type` (domestic league, domestic cup, continental, international, qualifier, friendly), `format`, `tier` (seeded from the reputation scale), `confederation`, `season_calendar` (affects form windows and congestion decay), `age_group` and `overage_rules`. Regime

facts that change over time (`has_var`, `bench_size_max`, per-half stoppage time) live in the temporal capability table (§3.4).

10.3 Promotion, relegation and transfers

When a team changes league, or a player transfers between leagues, the prior estimate carries over rescaled through Q_{league} rather than resetting to a flat average. `competition_history` is tracked per team and player. This is why a CompetitionBundle includes the feeder competition: a promoted team's carry-over needs the feeder league to have been fitted as a coherent whole, which cannot be done from isolated matches.

10.4 National teams

National teams differ from clubs in ways that matter: each plays roughly 8–15 matches a year, rosters change every window, a large share of matches are friendlies, tournaments are on neutral ground, and strength gaps are extreme. They get their own fit, linked to the club model through players (Figure 7).

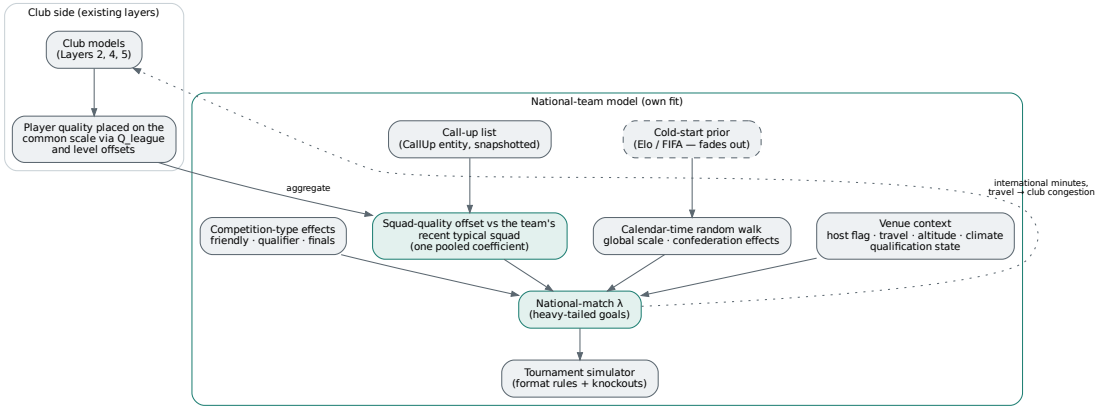


Figure 7: National-team model. Club-level player quality is placed on a common scale and aggregated into a squad offset; the national model has its own strength process, competition-type effects and tournament simulator.

Scope and fit. Teams flagged `is_national`, at senior and youth level, are modelled in their own random-walk fit rather than inside a club league, because the generating process differs.

One global scale. Unlike clubs, the national-team graph is well connected through qualifiers, nations leagues and tournaments. A single shared strength scale is used, with confederation as a hierarchical effect on the mean, home advantage and variance. This replaces the Q_{league} rescaling for this layer.

Calendar-time walk. The walk's variance grows with elapsed calendar time, not per match, with a longer effective window (about 8–12 years with decay). The "3–5 seasons" rule of §5.3 does not apply at about ten matches a year.

Squad prior and offset. The key link to the club model: called-up players' club-level quality, placed on the common scale through Q_{league} and level offsets, is aggregated into a squad offset relative to that team's recent typical squad. It reuses the lineup machinery of §7.5 with a single pooled coefficient across all national teams, because per-team data is too sparse. Elo or FIFA ratings serve only as cold-start priors that fade out, consistent with the prior-only rule.

Friendlies reverse the club rule. Because friendlies are a large share of national data, they are included with a competition-type effect: lower weight or extra variance, a rotation-aware

selection model, and their own scoring and home-advantage parameters.

Venue and context. A host flag plus travel and crowd-proximity covariates replace the neutral binary; altitude and climate matter more in tournaments. Importance extends to qualification state, distinguishing a dead rubber from a must-win.

Heavy-tailed goals. Mismatches break Poisson assumptions. Use a negative binomial or heavier-tailed goal distribution and a larger score-matrix truncation, with the truncation check of §14.5 enforced.

Call-ups. A temporal `CallUp` entity (player, team, window, status, `announced_at`) is snapshotted like player status. Nationality and eligibility edge cases go through `competition_eligible`.

Reverse link. International minutes and travel feed club-side congestion and injury risk.

Evaluation. Calibrate on qualifiers and nations-league-type matches (each tournament contributes only about 60 matches); report friendlies and competitive matches separately; use recent finals tournaments as small confirmation sets; benchmark against Elo and FIFA-ranking baselines (§14.4).

10.5 Youth, reserve and other team levels

The aim is to model every level for which data exists (Figure 8). The design differs from senior sides mainly because rosters are unstable.

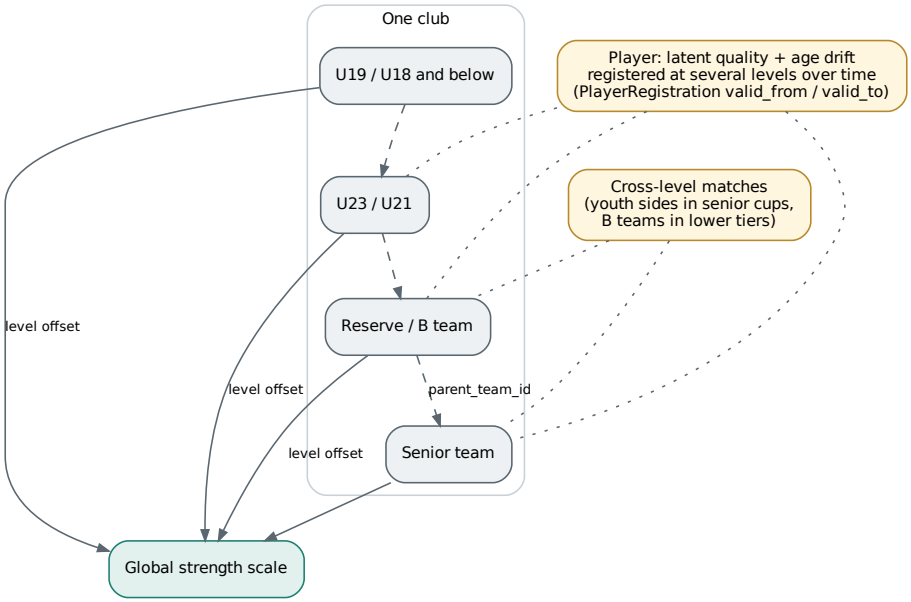


Figure 8: Team levels. Each level is its own competitor with a prior drawn from its parent and its own dynamics; players bridge levels and put them on one global scale.

- **Separate strength processes.** A club's U21 side is its own competitor with `parent_team_id` linking it to the club, not pooled into the senior walk. The prior flows from parent to child; the dynamics are the child's own.
- **Player-first strength.** A youth or reserve team's strength depends heavily on who plays: first-team players dropping down, loans, and overage slots. The lineup offset (§7.5) therefore does most of the work, with a small team-level random effect. This is the reverse of the senior balance.

- **Age dynamics.** Player quality is a function of age, with a development drift by age band and a cohort effect (`birth_date`). Youth rosters turn over every season as cohorts age out, so season transitions need a much larger variance reset than the soft reset of §5.3.
- **Bridging through players.** League scaling relies on cross-league matches (§10.1). For levels, players who move between them do the bridging: a player's latent quality is shared across levels with a learned offset per level, which places youth leagues on the global scale. Cross-level matches (youth sides in senior cups) bridge directly. This needs the temporal `PlayerRegistration` entity.
- **Rules metadata.** Age cutoffs and overage limits are on the competition (§3.2).
- **Noise.** Youth scoring is more variable with more blowouts: use the heavy-tailed distribution and larger truncation, and mark these projections low-confidence by default (§13.2).
- **Data.** The API appears to carry many youth competitions with thin lineup and statistics depth *[verify]*. Most will land on the baseline capability tier (results only: Layers 1, 2, 6, 7), which is acceptable under the degradation rules. They are pulled as a long-tail refit tier because extra leagues consume quota quickly.
- **Evaluation.** `team_level` is a standard segment; youth results are kept out of headline metrics; small leagues follow the minimum-observation rule and report "insufficient sample" where it applies (§14.2).

10.6 Sparse leagues, tiered refits and onboarding

- **Sparse leagues.** Low-data leagues lean more on the global prior and the reputation-derived Q_{league}; the confidence profile distinguishes reputation-prior-dominated from data-dominated (§13.2).
- **Refit tiers** are set by data volume, user interest and available API quota, not uniformly (§12.1).
- **Onboarding** is a defined, repeatable pipeline: scraper or adapter configuration, taxonomy entry, initial prior assignment, a **data audit** that populates the capability table, and assignment to a refit tier and bundle. It is a script with a checklist, because "as many leagues as possible" makes it a frequent operation (§16.6).

Part C — Serving and markets

From simulation to markets, inference cadence and retention, and explainability. Sections 11–13.

11. Simulation, markets and projection catalog

11.1 From simulation to markets

Given simulated (home, away) goal draws, or a score matrix truncated at a point such as 8–8 with tail mass folded into the last cell:

Market	Derivation
1X2	Sum matrix regions above, on and below the diagonal
Over/under (any line)	Cumulative total-goals probability; quarter lines average two adjacent lines
Asian handicap (any line)	Same quarter-line averaging; push handling by line granularity
Both teams to score	$P(\text{home} \geq 1 \cap \text{away} \geq 1)$
Draw no bet, double chance	Renormalised or pairwise sums of 1X2
Correct score	Direct matrix cells
Player props	Layer 5 draws conditioned on Layer 4 minutes and Layer 6 context
Corners, penalties, cards, fouls	Layer 5b/5c sub-model draws
Total substitutions (where offered)	Layer 4 substitution model
Half-time result, 1H/2H markets, HT/FT combined	Conditional half split of simulated full-time outcomes (§8.2); requires `has_half_split_stats`
Advancement, correct score after ET, shootout winner	Tie-resolution logic (§9)
Tournament outcomes (national and cup events)	Bracket Monte Carlo (§9.5)

Every market inherits full parameter uncertainty because it is computed from draws, not from a point-estimate λ plugged into a formula.

11.2 Coherence checks

Before any market reaches a dashboard: the correct-score matrix sums to one and to the 1X2 probabilities; over/under probabilities are monotonic across lines; handicap lines imply no contradictions; within each draw, half scores sum to full time and HT/FT probabilities sum correctly across the nine combinations; for knockouts, advancement integrates normal time, extra time and penalties without double counting. The score-matrix truncation is also checked for clipped probability mass (§14.5). These are cheap and catch normalisation bugs before a user sees them.

11.3 Dependence between sub-models

Sharing a posterior draw *index* across independently fitted sub-models creates no real correlation. The design therefore distinguishes:

- **Shared latents (preferred initial approach).** Goals, corners and cards condition on common Layer 3 covariates (style, congestion, importance), which preserves some real correlation without a joint fit.
- **Marginally calibrated only.** Where sub-models share no latents, each market is trustworthy on its own, and same-game combinations between them stay deferred until dependence is validated.

- **Joint or copula fit** is a longer-term option to be done when combination markets are built. No combination probability is claimed from shared draw indices alone.

11.4 Projection catalog

Rate and count projections marked Tier 1 or 2 are additionally available split by half where `has_half_split_stats` holds; this is a standing modifier and not listed as separate rows.

Match level

Projection	Tier	Layer	Notes
Score distribution matrix	1	L6/L7	Core simulation output
1X2, O/U, AH, BTS, DNB, DC	1	L7	Derived from the matrix
Correct score	1	L7	Direct cells
Total corners	1	L5b	Own rate model
Penalty markets	1	L5b	Own rate model
Total cards, red-card probability	1	L5c	Own rate model
Half-time result, HT/FT	1	L7	Conditional split of full time
Predicted formation (distribution)	1	L4	Structural input to lineups
Advancement; shootout winner	1	§9	Knockout matches only
Total fouls; total substitutions	2	L5c; L4	Secondary markets
Possession, big chances	2	Satellite regression	Context and explainability
Correct score after extra time	2	§9.2	Knockouts reaching ET
Own goals (total, team)	3	L5b	Rare; matrix accuracy

Team level (per team, per match)

Projection	Tier	Layer	Notes
Expected goals, decomposed (open play / penalty / set piece)	1	L6/L5b	Reported as a decomposition
Corners won and conceded	1	L5b	
Cards	1	L5c	Fouls are Tier 2
Projected squad (11, bench, out)	1	L4	Combines status feed, selection and formation
Shots (total, on target)	2	Satellite NB	Derived from λ and style
Goalkeeper saves	2	Satellite	GK props later
Tackles, interceptions, clearances	3	Aggregated players	Sum of player projections
Passes, distance, sprints	3	Descriptive / covariate	Not projected as outputs

Player level (per player, per match)

Projection	Tier	Layer	Notes
Availability status	1	L4	Deterministic where known
Start probability; expected minutes	1	L4	Core input to all props
Predicted role (slot code)	1	L4	Reshapes stat projections
Goals, assists	1	L5	Own goals excluded
Shots, shots on target; key passes, xA	2	L5	

Projection	Tier	Layer	Notes
Cards; fouls committed	2	L5c	Position- and history-conditioned
Tackles; interceptions, clearances; fouls drawn	2	L5	Distinct rate models
Composite rating	3	Meta-regression	Only once components are validated
WOWY impact, cohesion score	3	L4	Backlog internal terms (§20)

Player-level props are offered only where the competition supports them (§4.4).

12. Inference cadence and serving

12.1 Refit tiers

- **Full NUTS refits** are scheduled by tier: nightly for active high-interest leagues, weekly for the next tier, on demand or after each matchday for the long tail (including youth and sparse leagues). Posterior draws are stored as ArviZ netCDF in object storage.
- **Quota awareness.** A tier assignment is valid only if its data pulls fit the daily request budget (§4.2). The tier table is therefore set jointly by value, compute and quota.
- **Match-day serving** is posterior-predictive simulation off stored draws, with that day's lineup, context and override adjustments. Live NUTS per request is never run.
- **Parallelism.** Refits are largely independent per league. The global layer (§10.1) runs as a smaller joint step over summarised strength estimates, so cost scales with value rather than with the number of competitions onboarded.

12.2 Resimulation timeline

Every trigger fires an incremental resimulation off stored draws, never a refit (Figure 9).

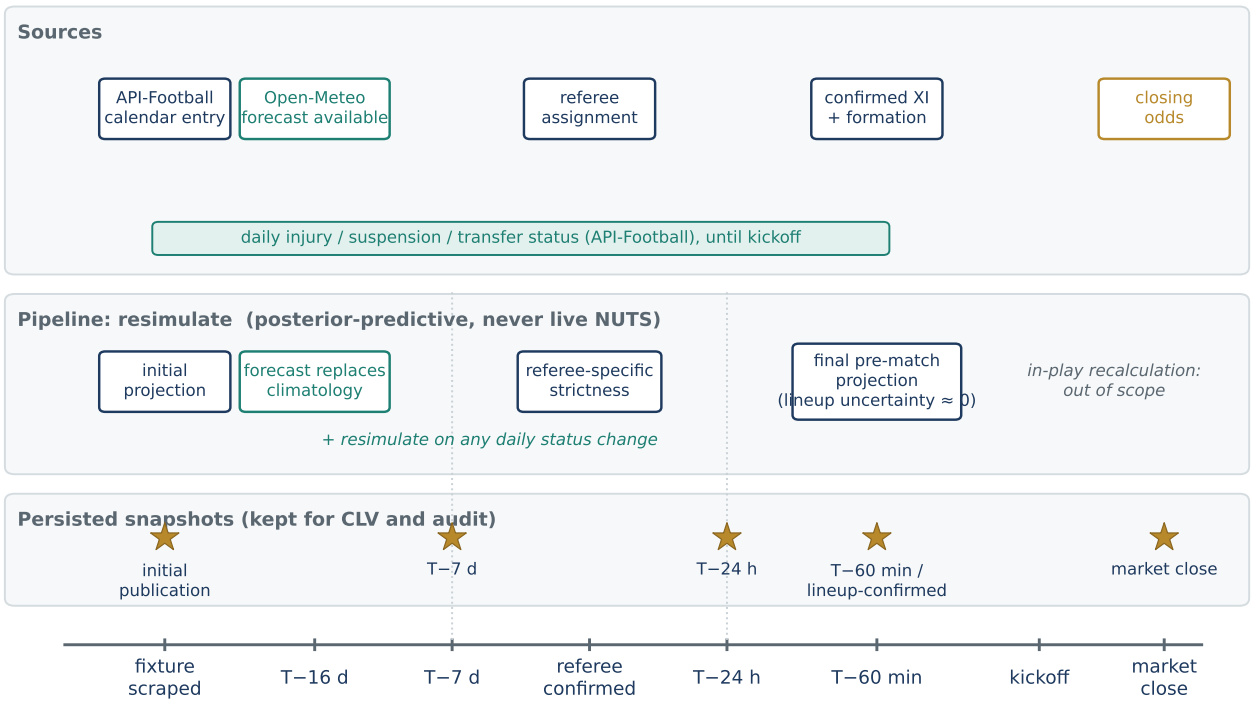


Figure 9: Match-day resimulation timeline and persisted snapshots.

Triggers: fixture first scraped (initial projection from statistical availability, most-likely formation, climatological weather, probabilistic referee pool); any daily status change; the weather forecast becoming available (about 16 days out); referee confirmation; and lineup confirmation about 60 minutes before kickoff, which replaces the projected-lineup distribution with the confirmed XI and brings lineup uncertainty to about zero. In-play recalculation is out of scope.

12.3 Retention and compute policy

- **Traces.** Retention follows champion/challenger status (§14.6): a retired run's trace is deleted after a grace window (for example 30 days); only the champion and active challengers keep traces indefinitely.
- **Resimulation snapshots.** Intermediate resimulations overwrite in place; permanent milestone snapshots are exactly the set CLV analysis needs: initial publication, T−7 days, T−24 hours, T−60 minutes (lineup-confirmed) and market close. The milestone set is tied to §15 and is not decided independently of it.
- **Raw payloads.** At least a year, compressed (§4.6), because re-fetching costs quota.
- **NumPyro migration trigger.** If one league's scheduled refit exceeds a wall-clock budget (for example 30–60 minutes), that league moves to the NumPyro path. The infrastructure follows observed cost, not assumed scale.
- **Compute control by gating.** WOWY, cohesion, half-split and interaction terms add parameters and refit time; failing the backtest bar for a league is also a reason not to pay their compute there.

12.4 Backup and recovery

Retention governs deliberate deletion; recovery covers accidental loss. Point-in-time recovery must be confirmed as enabled on the Postgres plan (it is sometimes an opt-in), object storage versioned or mirrored for anything not trivially re-derivable (a champion's trace cannot be regenerated without a full refit), and a recovery procedure documented and tested before real user data exists.

13. Explainability and confidence

13.1 Decomposition output

Because Layer 6 is a sum of log-space terms, a defined breakdown accompanies every projection, for example: league baseline + attack − defense + home advantage + style + lineup + congestion + manual overrides = $\log \lambda$. It is a debugging tool during development and a transparency feature for users.

13.2 Confidence profile

A single collapsed score is not used. Each projection carries named, independently visible components:

- **Historical data strength:** volume of relevant match, team and player history.
- **Source capability:** which capabilities apply for this competition, season and date, at what coverage ratio.
- **Lineup certainty:** projected versus confirmed.
- **Cross-league prior dependence:** whether the relevant Q_{league} is bridging-supported or reputation-prior-dominated; also whether a national or youth side is on its own sparse process.
- **Model stability:** time since last champion promotion and whether inference diagnostics are healthy for the segment.
- **Data freshness:** time since last successful refit and audit.

A headline label may be derived for compact views, but the components and the formula combining them must be visible and backtested; a label that cannot be decomposed is not trusted beyond its weakest component. A data-sufficiency threshold (matches or 90s) flags teams and

players as prior-dominated, and the interface distinguishes external-prior-dominated estimates (league reputation, player market value) from data-dominated ones.

Part D — Evaluation and governance

Backtesting, inference-quality gates, promotion rules, versioning, and the downstream edge module. Sections 14–15.

14. Evaluation protocol

14.1 Walk-forward harness

For each historical date t, refit and simulate using only data available as of t (the temporal contract, §3.3). Every market is scored against realised outcomes with Brier score, log loss, CRPS, and the ranked probability score (RPS) for ordinal markets such as 1X2, where a confident wrong-direction miss is worse than a wrong-adjacent one. Scores are tracked over time and segmented by league, market, model version, **team level** (senior, reserve, youth, national) and **confederation**; one aggregate hides regressions. This harness measures accuracy; the Edge/CLV module measures market disagreement and is separate.

14.2 Evaluation span versus training lookback

The evaluation span (how many seasons the backtest scores) and the per-prediction training lookback (§5.3) are different quantities. A five-season evaluation does not mean each prediction trains on five seasons: each as-of prediction obeys the production lookback. Required loaded history is warm-up history plus the evaluation span (a three-season evaluation with a three-season lookback needs about six seasons loaded). Early predictions with little history are genuine cold start; they are reported separately from headline numbers by default.

The span should reach a target observation count per metric, roughly 500–1,000+ for headline markets and much more for sparse ones (correct score, handicap lines, player props):

Competition type	Volume	Suggested span	Notes
Large domestic top flight	High	3–5 seasons (2–3 clears the threshold)	Covers a promotion and a manager-change cycle
Smaller leagues, cups	Low	5–8+ seasons	Some small cups never reach a robust sample; say so
Continental	Moderate	As much as available	Also the bridging source for §10.1
International tournaments	Very low	Several cycles	Add qualifiers and nations leagues (§10.4); confidence reflects persistent sparsity
Youth and reserve levels	Varies	Bounded by data reliability	Report separately; flag insufficient sample
Sparse long-tail leagues	Varies	Only where digitised data is trustworthy	Better short and reliable than long and noisy

14.3 Development window and locked holdout

Repeated testing on one evaluation period overfits that period. Three windows are therefore separated:

- **Development window:** used freely while iterating.
- **Locked holdout:** a rolling untouched period consulted only for a promotion decision, never to guide iteration.
- **Live forward period:** predictions made after the freeze are scored on arrival.

Proposed numeric gate for Phase 1b (to be confirmed or adjusted once Phase 1a gives a baseline spread): the production model must beat the Phase 1a static model on the locked holdout by at least **1.0% relative improvement in log loss on 1X2** and **0.5% in RPS**, with a paired 95% interval on the per-match score difference that excludes zero, with no tracked segment (league × market) worse than **1.0% relative** to the baseline. Thresholds are pre-registered before the holdout is opened, and recorded in the run log.

14.4 Baselines

The same harness scores simple baselines: a flat Elo-style rating, a non-hierarchical single-parameter Poisson, a "home win always" floor, and for national teams a FIFA-ranking or Elo baseline. If a baseline matches the full model's calibration in a segment, either the extra complexity is not yet earning its cost there or a bug is suppressing the model. The Phase 1a static Dixon-Coles is the permanent floor.

14.5 Inference-quality gates

Backtest improvement alone is not sufficient for a NUTS-sampled hierarchical model, because an apparent gain may be sampling noise. Before a challenger is eligible: convergence ($\hat{R}$ near 1), adequate effective sample size, near-zero divergences, posterior predictive checks (including **goal-count dispersion**), identifiability checks, prior-sensitivity checks, score-matrix truncation checks, and reproducible seeds. The diagnostics gate comes first and the backtest comparison second.

14.6 Champion/challenger promotion

A refit or model change is a challenger, never an automatic upgrade. Promotion requires: clearing §14.5; a win on the locked holdout with paired uncertainty intervals excluding zero; a minimum practical improvement threshold; no regression in any important segment beyond its limit (a deliberately accepted regression is recorded, not silently missed); and a record of every attempted challenger, including rejections with reasons, in the `model_runs` table. Status values: champion, challenger, rejected, retired.

14.7 Recalibration

The protocol must say what to do when a model is well-ranked but mis-calibrated. Where reliability diagnostics show systematic over- or under-confidence in a segment, an optional post-hoc recalibration step (for example isotonic or Platt-style mapping on the development window) may be applied, but it is itself a challenger: versioned, backtested on the locked holdout, and recorded in the manifest. Recalibration never replaces fixing a structural cause.

14.8 Reproducibility manifest

Every published projection carries: `projection_run_id`; component versions per layer (the Layer 2 version can differ from Layer 5's); global Q_{league} version; git commit; environment and lockfile hash; configuration hash; training-data snapshot hash; feature and source snapshot ids; override-set version; random seed; `generated_at`; `as_of`. "Why did this projection say 64%" must be answerable months later. Experiment tracking starts as a Postgres table; an experiment-tracking service is added only if run volume demands it.

15. Edge/CLV module

Explicitly not part of the model. Odds never enter Layers 1–7. The module ingests odds, de-vigs them, and compares them with the model's market probabilities per match, market and league.

15.1 Definitions

- **Probability edge:** model probability minus de-vigged market probability (a diagnostic).

- **Expected value:** model prob × decimal odds − 1, the primary edge metric because it accounts for payout.
- **Closing line value (CLV):** whether the odds available when the projection was made beat the closing price. The close is the most information-efficient price, so CLV is a lower-variance, faster validity signal than realised ROI, which is dominated by variance over practical sample sizes. Both are tracked; CLV leads.

15.2 Data requirement

CLV needs two distinct captures: the odds at decision time and the close. The milestone set of §12.3 (initial, T−7d, T−24h, T−60m, lineup-confirmed, market close) supplies this. Per snapshot: bookmaker, line, odds timestamp, projection timestamp, de-vig method, and how the consensus was built (which books, how weighted). **Forward capture starts in Phase 0/1**, because pre-match odds history cannot be reconstructed later; the API's pre-match odds are captured on the milestone schedule from the first day the pipeline runs, even though the comparison module is built much later *[verify depth and retention]*.

15.3 Segmentation

Dimension	Why it matters
Market type	Niche and prop markets get less sharp attention than 1X2
League, competition, team level	Efficient top leagues versus less-covered ones; youth markets are thin or absent
Odds source	Edge against a soft book is a weaker claim than against a sharp or consensus line
Favourite versus underdog	Favourite-longshot bias: check that edge is not a rediscovered known inefficiency
Time to kickoff	Does edge decay toward the close (speed) or persist (a different view)?
Confidence bucket	A validity check on the confidence profile itself

15.4 Guardrail

Before treating any apparent edge as real, cross-check calibration in the same segment against the walk-forward harness. A segment with strong "edge" but weak or unvalidated calibration is very likely model error disagreeing with a calibrated market. Edge and calibration are always read together.

15.5 Reporting

A dedicated edge analytics view (§16.4) and an automated weekly or monthly report: CLV trend by segment, EV distribution, edge-versus-confidence correlation, and sample sizes. Confidence intervals accompany every figure, and thin segments are flagged rather than shown with unqualified headline numbers.

Part E — Product and platform

The dashboard views, the technology stack and its cost, and the legal, privacy and continuity items. Sections 16–18.

16. User interface

16.1 Views and build order

The product is a small set of views serving different jobs; mixing them (for example admin information inside the match view) makes the primary analytical view worse. Thirteen views are specified, built in three waves.

View	Job	Audience	Wave
§16.1 Fixture and competition browser	Find a match	Everyone	1
§16.2 Match projection dashboard	Assess one match in depth	Analysts	1
§16.3 Pre-match override interface	Change something about an upcoming match	Analysts	1
§16.5 Model ops (lean)	Is the model healthy	Operator	1
§16.11 Backtest explorer	Investigate calibration by segment	Operator	1
§16.4 Edge analytics	Find and validate edge in aggregate	Analysts, later commercial	2
§16.6 Competition management and coverage	Onboard and monitor competitions (includes the coverage map)	Operator	2
§16.8 Team profile; §16.9 Player profile	Inspect strength trajectories, rates	Analysts	2
§16.10 Knockout and tournament tracker	Advancement across a bracket	Analysts	2 (with Phase 3d)
§16.12 Override impact review	Did an override help	Operator	3
§16.14 Fixture congestion calendar	Anticipate rotation	Analysts	3
§16.7 Settings, notifications, audit	Alerts, audit log, digest	Operator	1 (minimal)

The coverage map (formerly a separate view) is folded into §16.6. Every credible interval and confidence component required below is non-negotiable in every view that shows a probability; a clean interface that quietly discards the uncertainty is a bug.

16.2 Fixture and competition browser (§16.1)

Competition-grouped fixture lists, not a flat calendar. Per-competition header with a capability badge and refit-tier freshness indicator. Per fixture: teams, localised kickoff, a headline probability, and the confidence badge. Filters by competition, date, confidence tier, team level, and, once data exists, “has edge”. Knockout fixtures show leg and aggregate context.

16.3 Match projection dashboard (§16.2)

The centrepiece, in this order: header strip (teams, competition, kickoff countdown, venue, weather, referee or pool indicator, capability badge); confidence components; headline markets with credible intervals and a correct-score heatmap; the decomposition strip as a waterfall; squad and lineup panel on a formation diagram using real slot codes, bench, and the out-list with reasons; player projections table with assigned role; half-split toggle where supported; a knockout panel where applicable; a collapsed edge strip; and a persistent override entry point. Every projected number is shown in a monospace style distinct from labels. The visual treatment (dark instrument panel, monospace numerals, waterfall as the signature element) is the reference for all other views.

16.4 Pre-match override interface (§16.3)

A drawer scoped to one scheduled match, covering both mechanisms (§8.5): structural edits (availability, formation, lineup slots and roles, referee, weather, venue, `competition_eligible`, call-up status) and parametric scale factors. Overrides apply to that match only by default, trigger immediate resimulation, and are logged with who, when, what and why. The log is also a diagnostic feedback loop: a parameter consistently overridden in one direction signals systematic model bias, which §16.12 mines.

16.5 Edge analytics (§16.4)

CLV trend as the headline chart; segmentation controls matching §15.3; every segment with sample size and interval, with thin segments visually de-emphasised; a hard-wired calibration cross-check beside any apparent edge; and an EV distribution histogram so outliers are visible.

16.6 Model operations (§16.5)

Champion and challenger status, backtest score trends by segment, naive-baseline comparison, data-quality and drift alerts in one feed, an override-mining panel, and storage, compute and **API quota** usage per tier. The lean first version covers champion status, score trends and the alert feed.

16.7 Competition management and coverage map (§16.6)

An onboarding checklist per competition (adapter configuration, detected capability flags as a confirm-or-override list, initial prior, refit tier, bundle). A competition × capability grid (lineups, player stats, xG, corners, cards, half-split) with coverage ratios and last-audited dates, refit freshness, bundle membership, and Q_{league} support status (bridged versus reputation-dominated), which shows where continental or international data would add the most calibration value.

16.8 Settings, notifications and audit (§16.7)

Alert routing and severity thresholds; a consolidated, filterable audit log of overrides and promotions; minimal authentication for a small known group; and a daily digest summarising notable edge, confidence downgrades and recent override activity. Persistent “informational projections, not betting advice” language lives in the footer (§18.1).

16.9 Team profile and player profile (§16.8, §16.9)

Team: a strength trajectory chart with the interval widening at season transitions and manager changes; current parameters (attack, defense, home advantage, form, level and Q_{league} context); recent normal-time form; and manager context with formation tendencies; for national teams, the squad-offset history. *Player:* fitted rates versus the cold-start prior (how far real data has moved the estimate); position-code history; ramp status; and, once those backlog terms exist, WOWY and cohesion partnerships. Player pages for under-18s follow the privacy rule in §18.3.

16.10 Knockout and tournament tracker (§16.10)

One card per tie showing legs, aggregate, and the advancement breakdown (normal time, extra time, penalties separately), with tie states made explicit (not started, leg 1 complete, single leg reaching ET), away-goals context where valid, and, for tournaments, group and bracket probabilities.

16.11 Backtest explorer (§16.11)

Interactive slicing by league, market, metric, evaluation span, team level and confederation; the naive-baseline overlay on every chart; and a per-segment breakdown table that makes the regression warning of §14.1 actionable.

16.12 Override impact review (§16.12)

For each played, overridden match: the projection with the override, without it, and the result. Per match the claim is a change in realised proper score ("improved the score for this observed result"), never "improved calibration", which is a property of a collection. Recurring-pattern detection flags consistent, consistently harmful override types only on aggregated samples with intervals, and feeds the challenger cycle.

16.13 Fixture congestion calendar (§16.14)

Teams as rows, the next two weeks as columns, fixtures colour-coded by competition type, plus a plain-language list of teams heading into heavy rotation risk, surfacing the forward-looking congestion covariate as something a person can act on.

17. Technology stack and cost

17.1 Stack

Pricing is a snapshot (mid-2026); confirm current rates before committing.

Concern	Choice	Why
Relational data	Postgres (Neon)	Relational integrity and as-of queries; scale-to-zero pricing; branching for migration tests
Traces and raw payloads	Cloudflare R2	Large binaries; zero egress matters because every resimulation reads traces
Orchestration	Self-hosted Prefect, or scheduled jobs initially	A single bundle may need only cron-style jobs; adopt the orchestrator when dependencies justify it
Modelling	PyMC, ArviZ; NumPyro for scale	Open source; NumPyro on observed cost trigger (§12.3)
Data wrangling	Polars	Multi-threaded, lazy evaluation
API and serving	FastAPI, SQLAlchemy/SQLModel, Alembic on Railway	Developer speed over edge latency
Frontend	Next.js self-hosted alongside the API	No per-seat hosting bill for an internal tool
Auth	Supabase Auth or Clerk (free tier)	Small, known user group
Packages	uv	Existing choice
Experiment tracking	`model_runs` table	No extra service yet
Data provider	**API-Football Pro**, upgradeable	Replaces scraping and the proxy line; quota-managed (§4.2)
Supplements	Transfermarkt (values, contracts); Open-Meteo	Narrow roles

Concern	Choice	Why
CI/CD	GitHub Actions	Software tests, plus model backtest before a challenger is eligible
Alerting	Slack webhook or similar	Data-quality, drift, budget and quota alerts

Compute (scraping, refits) and serving (API, frontend) remain separate deployables.

17.2 Cost

Stage	Components	Approximate monthly cost
Start: one bundle, low traffic	Database free tier; object storage free tier; hosting about $5; **API-Football Pro (a low two-digit monthly fee; confirm)**; refits on the same small host	roughly $25–30
Growth: many leagues, daily refresh, dashboard use	Database $15–25; storage $5–10; hosting $20; API plan (Pro, or the next tier if quota binds); **refit compute (see below)**	roughly $70–150+

Refit compute is a real line item. A nightly NUTS refit of several leagues on a small shared host will not be adequate for the full build. The cost model must state: expected wall-clock per league refit (measured in Phase 1b), where it runs (a larger on-demand worker is the likely answer), and monthly cost at the tiered cadence of §12.1. The NumPyro/GPU path is added when a league breaches its time budget and is costed then. The earlier "$5 per month to start" figure excluded the data provider and refit compute and should not be used as a planning number.

The scraping-proxy budget line of the earlier draft is removed.

17.3 Budget and quota alerting

Reuse the alert webhook to flag real spend deviating from the estimates, **and API quota usage** (daily consumption against the plan, with the upgrade trigger of §4.2). A runaway job should surface within days, not at the end of a billing cycle.

18. Legal, privacy and continuity

These items require outside expertise (a lawyer, an accountant). They are open items the plan must account for and are not resolved by anything in this document.

18.1 Regulatory exposure

Although odds never enter the model, the outputs (1X2, over/under, handicap, correct-score probabilities) are functionally a prediction or tipster service. Whether providing probabilities, without taking bets, triggers licensing depends on jurisdiction and on positioning and monetisation. Malta hosts an active gaming regulator, so a lawyer familiar with the Maltese framework should be consulted before any public launch or monetisation.

Youth markets. Many regulators and bookmakers restrict or avoid betting markets on youth events. Whether and how youth and under-18 projections may be exposed in any commercial product belongs in the same legal conversation; the default for youth data is internal-only analysis. The Edge/CLV module will in practice have little or no youth odds data.

Responsible-use messaging is built in from the start: persistent "informational projections, not betting advice" language in the interface, and, if the product becomes public, a link to problem-gambling resources.

18.2 Data licensing

- **API-Football.** A licensed API removes most of the scraping-era exposure, but the provider's terms on *commercial redistribution* of data and derived outputs must be read before any revenue-generating use, and the plan tier may constrain use *[verify]*.
- **Other sources.** Transfermarkt market values and contract dates (scraped or otherwise obtained) and the league-reputation scale have their own terms and are covered by the same review. **The review also covers any odds source** (for example a site scraped for historical closing odds), which the earlier draft omitted.
- **Graduation path.** Enterprise data providers (Opta/Stats Perform, Sportradar) are quote-only and expensive (published third-party estimates run from hundreds to over ten thousand dollars a month and should not be used for planning without a real quote); developer-focused providers sit between. The current provider is itself a stepping stone. Treat licensed enterprise data as a planned graduation point once revenue justifies it.

18.3 Privacy

Once the dashboard has user accounts beyond a small known group, GDPR applies by default for an EU-based operation: a documented retention policy, a path for deletion requests, and a privacy policy. **Minors.** Player profiles and projections for under-18s carry extra sensitivity: decide what stays internal and what any public product shows, minimise stored personal data (the schema needs birth date and registration history, not more), and avoid surfacing individual youth-player projections publicly by default.

18.4 Documentation and bus factor

Keep the README current as code is built. Record Architecture Decision Records for any departure from this document, with dated reasoning at the time of the decision; the switch to API-Football, the lineup-offset design (§7.5) and the half-split rule (§8.2) are the first three. This keeps the document and the system from silently drifting apart.

18.5 Incident and postmortem habit

When the pipeline breaks silently or the model makes an embarrassingly confident wrong call, write a few lines: what happened, why (root cause), and what changes (a monitor, a backtest segment, a process). A dated note in the repository is enough; the value is in always closing the loop.

Part F — Delivery

The build roadmap, timeline, traceability, and the backlog of deferred items. Sections 19–20.

19. Roadmap

19.1 Scaling strategy

- **One bundle first.** Terminology is fixed: a *Competition* is a single entity; a *Competition-Bundle* is a group maintained together. Phase 1 covers one bundle of two competitions (the English top flight and its immediate feeder, whose promoted-team carry-over needs the whole league fitted).
- **Warm-up plus evaluation.** Required loaded history equals warm-up history plus the evaluation span (about six seasons for a three-season evaluation with a three-season lookback). Configuration, not prose, holds: `backfill_start`, `evaluation_start`, `evaluation_end`, the production lookback policy, a minimum warm-up count, and whether cold-start predictions are reported separately (they are, by default). With the API as source, the backfill is also a quota event (§4.2).
- **Top-tier, data-rich bundle first,** so the full model is validated before the degradation paths are tested on a sparse league as a deliberate second case.
- **No cups at the start.** Knockout logic (§9) is new modelling work and is validated separately after the round-robin core.
- **International and levels come in layers** (§19.3), not at the start.

19.2 Getting started: the first weeks

1. **Accounts and environment.** Database, object storage, hosting, repository; package management; a connection test to each.
2. **Minimal schema** (the Phase 0 subset of §3.5), migrations from the first table, with `team_level`, `parent_team_id`, `PlayerRegistration`, the temporal contract and capability table included now, because retrofitting them is a redo.
3. **API-Football adapter and data audit.** Competition discovery, fixtures and results, with normal-time, extra-time and penalty scores separated from the first line of code; populate the capability table from real payloads (§4.4); record quota usage from the start; write tests for payload parsing.
4. **Start snapshot capture immediately** (a cron-style job, no models needed): daily player-status snapshots, fixture-calendar snapshots, and pre-match odds snapshots on the §12.3 milestone schedule. These cannot be recovered later, and capturing them from the beginning gives Phases 3 and 7 a real history.
5. **Backfill** the bundle (about six seasons) and fix whatever breaks.
6. **Phase 1a baseline** (below), then the backtest harness immediately, before any "real" model, so the measuring stick exists before there is anything to measure.
7. **Move the build into the repository**, with agentic coding sessions for scaffolding, schema work, adapters and model iteration.

19.3 Phases

Phase	Content	Hard gate before moving on
0 Foundation	Phase 0 schema (§3.5), taxonomy, capability table, temporal contract, manifest tables, storage; **snapshot capture running**	Schema migrates cleanly; first snapshots arriving

Phase	Content	Hard gate before moving on
1a Baseline	Static non-hierarchical Dixon-Coles, 1X2/OU/BTS, walk-forward harness end to end. Benchmark-only (exempt from the posterior principle)	Pipeline runs end to end
1b Production team model	Priors, random walk, AR(1) form, home advantage, **xG auxiliary signal and manager-change widening (where the audit shows data)**; posterior simulation with intervals	§14.5 gates, then the pre-registered threshold of §14.3 on the locked holdout
Feasibility register	Field × source × coverage entries for all planned covariates	Every Phase 2/3c covariate has a verified entry
2 Context	Style (API-provided only), venue, weather, referee, congestion, importance	Each covariate re-backtested and kept only if it improves calibration
3 Squad and player	In steps: (a) status feed and squad lists; (b) availability and selection validated standalone; (c) formation and slot-code roles, Manager entity; (d) **lineup offset in the likelihood vs serving-only** comparison; (e) feedback loop wired into Layer 6	Each step validated before the next
3b Goals, discipline, half-split	Total-first decomposition, discipline, substitutions, half-split with the full-time-primary rule (§8.2)	Each sub-model backtested as an allocation
3d Knockouts	Format rules, ET, shootouts, two-legged ties; first cup or continental knockout stage	Validated separately from the round-robin core
4 Explainability and monitoring	Decomposition output, confidence profile, drift monitoring, coherence checks (incremental, alongside 3)	Ongoing
5 Interface (wave 1, then 2, 3)	Views per §16.1	Term list stable before building against it
6 Scale-out	Further bundles incl. one continental competition for Q_{league} bridging; **youth and reserve levels as a long-tail wave**	Data audit run honestly per competition
7 Edge/CLV module	Comparison, de-vig, segmentation, calibration cross-check, reports (capture has been running since Phase 0)	Calibration cross-check wired before any edge is trusted
8 Second sport	Plugin abstraction proven (Appendix G)	
International	**v0** team-only from results, soon after 1b; **v1** squad offset after Phase 3; **v2** tournament simulator after 3d	Each beats the Elo baseline on the locked holdout

The advanced-signal items formerly in Phase 3c (WOWY, cohesion, workload risk, style-matchup, contract, kickoff time) are now an **experiment backlog** (§20.2), run one at a time through the baseline discipline after Phase 3, not scheduled work.

19.4 Timeline

Assumptions: solo operator, 10–15 hours per week of part-time effort. Agentic coding speeds typing but not validation loops, convergence debugging or waiting on a covariate's re-backtest. Estimates are stated in hours with a buffer, as targets to check against rather than commitments.

Milestone	Target effort	Cumulative (at 10–15 h/week)
Phase 0 + 1a: accounts through baseline and harness	40–70 h	3–6 weeks
Phase 1b: production team model	60–100 h	6–13 weeks
Feasibility register + Phase 2	40–70 h	9–17 weeks
MVP milestone: 1b + nightly serving loop + match dashboard (§16.2)	140–240 h	**about 11–18 weeks**
Phase 3 + 3b	120–200 h	5–8 months
Phase 3d, International v0	50–90 h	6–10 months
Phase 5 wave 1 and 2	80–140 h	8–12 months
Phase 6 first wave, Phase 7	80–140 h	10–16 months

The full scope is on the order of **450–800+ hours**, and a **buffer of 25–40%** should be added to every row. The MVP is the realistic headline: from that point the system produces daily value and everything after is refinement. Phase 8 is archetype-dependent (about 2–4 months for another rate-based invasion sport; 3–5 for nested set or combat sports; 6–9+ for ball-by-ball sports) and open-ended scale-out continues indefinitely.

19.5 Traceability

Each model term, with its phase, required capability and feasibility status, is tracked in Appendix C, so a term that depends on an unavailable field cannot silently survive in the plan.

20. Backlog and deferred items

20.1 Deferred by design

- **In-play and live recalculation:** a different engineering problem (streaming state, subsecond latency, live feed cost). Kept out so it does not pull architecture decisions in the wrong direction.
- **Same-game combination markets:** deferred until dependence between sub-models is validated (§11.3).

20.2 Experiment backlog (run one at a time, after Phase 3)

Each is expected to earn its place or shrink toward zero; both outcomes are a successful use of the harness. None has scheduled phase time.

- WOWY marginal player impact; lineup cohesion; workload-trend injury risk (needs physical-load data); dynamic home advantage via attendance (needs attendance data); contract situation; kickoff time of day; altitude and forward-looking congestion refinements; new-signing ramp tuning.
- **Style-matchup interaction** (a heavily shrunk product of the two sides' style covariates, tested by structured residuals by pairing type rather than team identity), and its manager-conditioned extension.

Why raw head-to-head is not a feature. Head-to-head record is mostly already captured by time-varying strength, is a tiny noisy sample, reflects past squads, and is a documented pitfall in goal-model forecasting. Manager-versus-manager records fail the same way, more strongly. The defensible form is the style interaction above.

20.3 Not planned

Tier 3 outputs that drive no market: the composite player rating, passes and pass accuracy, possession as a standalone product, and other purely descriptive statistics.

20.4 Other deferrals

Sport archetypes beyond the first second sport (Appendix G); public-facing youth projections (§18); a fully joint or copula model of sub-models.

Appendices — Reference material

Detailed diagrams, schema subset, traceability, quota budget, feasibility register, change history, multi-sport abstraction and glossary.

A. Detailed model DAG

The full pipeline with every sub-model and data source. It supersedes the earlier detailed DAG: the Layer 6–7 box no longer claims a shared draw index preserves correlation (§11.3), the feedback arrow from evaluation is limited to backtest results, and the lineup offset enters once.

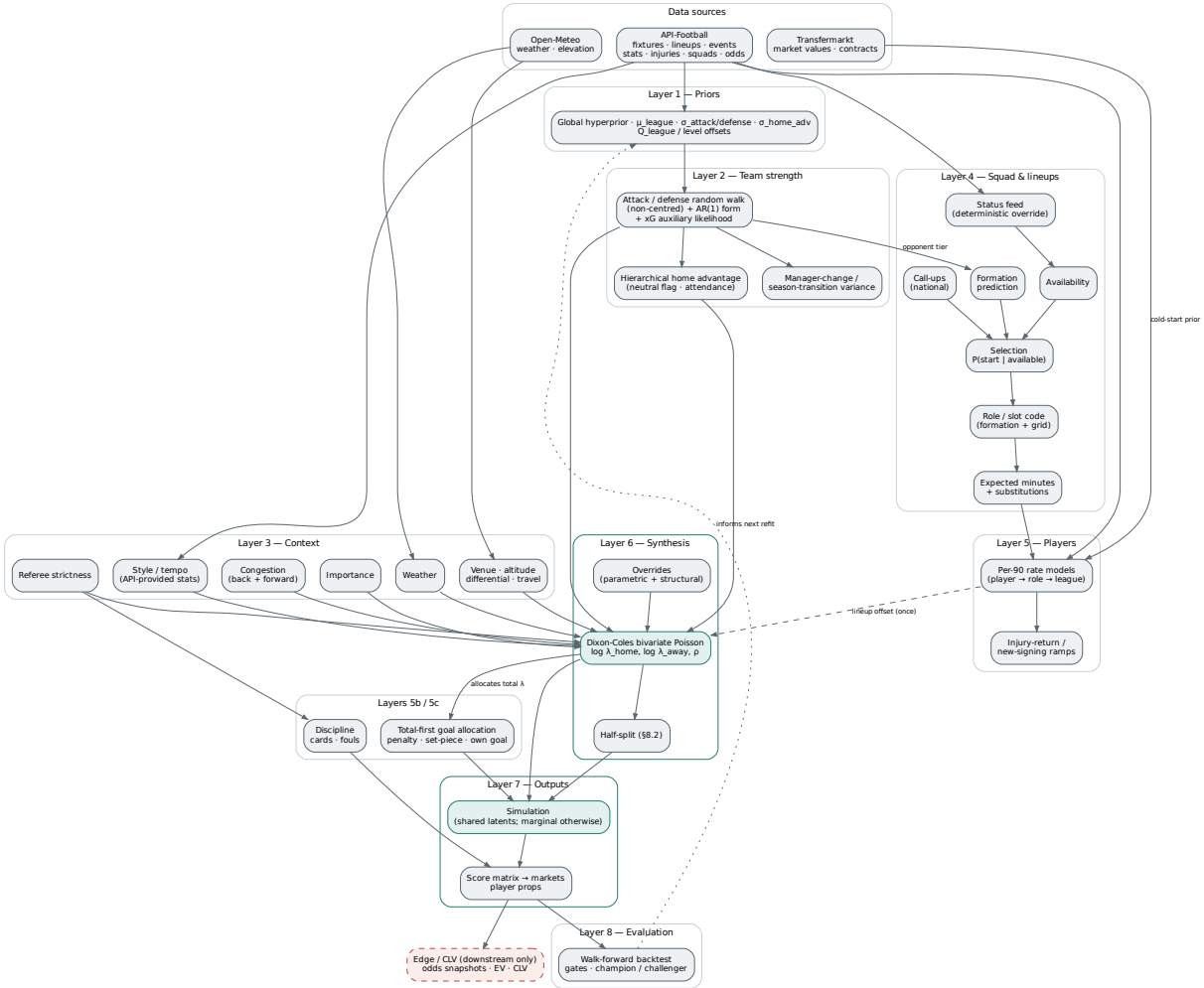


Figure 10: Detailed model DAG.

B. Phase 0 and end-state schema

Entity	Phase 0	Added later	Trigger
Competition, CompetitionBundle, Season	yes		
Team (`team_level`, `is_national`, `parent_team_id`)	yes		
Match (normal-time, extra-time, shootout fields; temporal fields)	yes		
Player, PlayerRegistration, PlayerStatusSnapshot	yes		
CompetitionSourceCapability	yes		
Override tables	yes		
ModelRun, ProjectionRun, MarketProjection	yes		
OddsSnapshot	yes (capture only)	comparison in Phase 7	
Venue, Referee		Phase 2	Context layer
Lineup, Manager, ManagerTenure		Phase 3	Squad layer
CompetitionRound, KnockoutTie		Phase 3d	Knockout logic
CallUp		International v1	Squad offset
Competitor, Period, generalised MatchEvent		Phase 8	Second sport

The end-state diagrams are Figures 2–4 in §3.5.

C. Traceability matrix

Model term	Phase	Required capability	Source	Status
Static Dixon-Coles baseline	1a	results only	API-Football	verify coverage
Random walk, AR(1), home advantage	1b	results only	API-Football	verify coverage
xG auxiliary likelihood	1b	`has_xg`	API-Football	verify coverage per league-season
Manager-change widening	1b	manager history	API-Football (coach data)	verify
Neutral flag, host flag	1b	venue and stage data	API-Football	verify
Style covariates	2	`has_shot_stats`	API-Football	reduced set
PPDA, directness	not planned	event or tracking data	none identified	likely unavailable
Weather, altitude	2	venue coordinates	Open-Meteo	available
Referee strictness	2	`has_referee_history`	API-Football (names)	verify
Congestion (back and forward)	2	fixture calendar	API-Football	available

Model term	Phase	Required capability	Source	Status
Status feed, availability	3a–b	injuries, squads	API-Football	verify return-date depth
Formation, slot codes	3c	`has_lineups`	API-Football (grid)	verify
Player rates, ramps	3	`has_player_match_stats`	API-Football	verify
Market-value cold-start prior	3	market values	Transfermarkt	available (licensing review)
Lineup offset in likelihood	3d	`has_lineups` history	API-Football	verify depth
Goal decomposition (total-first)	3b	event-level goal types	API-Football	verify
Discipline, corners	3b	`has_cards_data`, `has_corners_data`	API-Football	verify
Half-split	3b	`has_half_split_stats`, event minutes	API-Football	verify
ET and shootout models	3d	normal-time/ET/pen scores	API-Football	verify
International v0–v2	after 1b / 3 / 3d	results, call-ups	API-Football	verify call-up data
Youth and reserve levels	6	results (lineups rarely)	API-Football	verify coverage
Workload injury risk	backlog	`has_physical_load_data`	none identified	blocked
Attendance-based home advantage	backlog	`has_attendance_data`	unknown	blocked
WOWY, cohesion, contract, kickoff time, style-matchup	backlog	lineups, contracts	API-Football, Transfermarkt	experiment
CLV	7 (capture from 0)	odds snapshots	API-Football, others	verify retention

D. API-Football request budget

All numbers are illustrative planning assumptions and must be replaced by measured usage from response headers *[verify]*.

Item	Assumption	Calls
Plan quota	Pro	7,500 per day
Backfill, 2 leagues, 6 seasons	about 5,600 matches; if detail (events, lineups, statistics, player stats) needs about 4 calls per match	about 22,000 calls: roughly 3–4 days of quota
Backfill with batched id lookups	up to 20 fixtures per call, if supported	a few hundred to a few thousand calls
Steady state, daily results	one date query per league	tens per day
Match-day polling, tier-one league	lineups, status for about 10 matches, several polls each	on the order of 100–300 per league per match-day
Daily status feed	injuries and transfers per league	tens per league
Long-tail and youth leagues	date queries, no lineups	a few per league per day

Rules. Reserve a fixed share of the daily quota for match-day polling; run large backfills in

off-peak windows or against a temporarily upgraded plan; upgrade when planned steady-state use is sustainably above about 70% of quota; record actual usage per job so the tier table (§12.1) is built from measurements.

E. Source-feasibility register (template)

One row per field. The register is completed in the data audit (§4.4) before the dependent sub-model starts.

Column	Meaning
Field	e.g. formation grid, xG, injuries return date
Source, endpoint	Provider and call
Historical availability	From which season; gaps
Competitions covered, coverage %	Per competition and season; from real payloads, not declared flags
Update cadence	How soon after the event
As-of reconstructable	Can a past value be reproduced, or must it be captured live
Licensing status	Terms for the intended use
Fallback	Alternative source or degraded behaviour
Required phase	Where the dependent work sits

F. Change history and decision log

Corrections folded into the main text. The first draft carried these as inline notes; each is now stated cleanly where it belongs.

Topic	Original statement	Correction	Now in
Global hyperprior	Pools leagues and makes strength comparable	It lets sparse leagues borrow strength only; relative quality needs Q_{league}, bridging and priors	§5.1, §10.1
Goal decomposition	Penalty, set-piece and own-goal rates added to total	That double-counts; the design is total-first allocation	§8.3
Player and team feedback	Player quality shapes team λ, projections conditioned on λ	One-directional, allocated once, shares normalised	§7.5
Joint simulation	Same draw index preserves correlation	Only shared latents or a joint fit do	§11.3
Snapshot retention	Keep only final lineup-confirmed projection	Keep the milestone set CLV needs	§12.3, §15.2
CLV data	Ingest closing odds	Needs decision-time and closing captures	§15.2
Promotion rule	Promote if it beats champion	Add locked holdout, paired intervals, thresholds, regression limits, rejected-run log	§14.3, §14.6
Inference quality	Backtest sufficient	Diagnostics gate comes first	§14.5
Reproducibility	Commit and config hash	Full manifest	§14.8
Capability flags	Static booleans on Competition	Temporal fact table	§3.4
Player-prop fallback	Permanent market-value fallback	Suppress props; coarse lineup adjustment only	§4.4
Override review wording	“Improved calibration” per match	Per match: change in realised proper score	§16.12
Schema terminology	“Multi-sport-ready”	Multi-sport-migratable	§3.5, App. G

Topic	Original statement	Correction	Now in
Confidence	Single derived number	Named components	§13.2

Decisions in this revision (ADR seeds)

1. Replace Sofascore scraping with API-Football; keep Transfermarkt and Open-Meteo narrowly.
2. Lineup offset inside the training likelihood, not serving-only (§7.5).
3. Full-time score matrix is primary; halves are conditional splits of it (§8.2).
4. National teams have their own fit linked through players (§10.4).
5. Every team level modelled where data exists, with `team_level` and `parent_team_id` (§10.5).
6. Snapshot capture begins in Phase 0.
7. Advanced-signal items moved to an experiment backlog.

G. Multi-sport abstraction

Target roster (order undecided): basketball, tennis, baseball, hockey, NFL, Aussie rules, boxing, cricket, darts, esports, MMA, rugby league, rugby union, snooker, volleyball. These do not share one generating process, so the plugin interface stays constant while the engine varies by archetype.

Archetype	Sports	Core engine	Reuse from football
A. Rate-based team invasion	Basketball, hockey, NFL, Aussie rules, rugby league, rugby union	Poisson/NB or margin regression per period; half-split generalises to N periods via a Dirichlet split	Highest: team strength, lineups, period-split machinery
B. Nested set, leg or frame	Tennis, volleyball, darts, snooker	Point or game win probability propagated through a Markov chain	Hierarchical strength pooling; scoring engine is new
C. Combat and decision	Boxing, MMA	Competing risks: $P(\text{win}) \times P(\text{method} \mid \text{win}) \times P(\text{round} \mid \text{method})$	Rate-stat pooling
D. Sequential state machine	Cricket, baseball	Ball-by-ball or at-bat micro-simulation	Least: layering concept only
E. Map and round structure, fluid rosters	Esports	Closest to B for win probability, with football-like roster volatility	Lineup machinery; each title needs its own sub-plugin

Generalised entities (added when sport #2 is onboarded). `Competitor` is a polymorphic wrapper (team, player or pair) that `Match` references; `Period` generalises halves to quarters, innings, sets, rounds, legs, frames or maps; `MatchEvent` gains a `points` field and a sport-defined `event_type`. `Manager` stays top-level (and is team-sport-only). Each sport needs its own capability-flag set rather than reusing football's.

Migration path. `Match.home_team_id` and `away_team_id` become `competitor_1_id` and `competitor_2_id`: add the Competitor table, backfill one row per existing Team, add the new foreign keys alongside the old, populate them, then drop the old columns once every read path is migrated. This preserves history without a destructive rewrite; the script is planned now and run before Phase 8.

Plugin interface. `fit()`, `simulate()`, `project_markets()` against the generic schema. Football's Dixon-Coles, offside-driven statistics and formations live in the football plugin. Sequencing: an Archetype A sport is the cheapest proof of the abstraction; B and C need new engines; D is the heaviest and is explicitly the lowest priority.

H. Glossary and notation

Term	Meaning
As-of	Using only information known at a given timestamp
Bridging match	A match between teams from different leagues or levels that identifies relative strength
CLV	Closing line value: decision-time odds versus closing odds
Competition / Bundle	A single competition / a group onboarded together
Dixon-Coles	Bivariate Poisson goal model with a low-score dependence correction ρ
ESS, $\hat{R}$	Effective sample size; convergence diagnostic
EV	Expected value: probability × decimal odds − 1
λ	Expected goals rate for a team in a match
NUTS	No-U-Turn sampler for Bayesian inference
PPC	Posterior predictive check
Q_{league}	League-quality parameter placing leagues on one scale
RPS	Ranked probability score, for ordered outcomes
Slot code	Position code for a formation slot (RB, RWB, CB, CM, ST, …)
Total-first	Goal sub-models allocate a fitted total rather than adding to it
WOWY	“With or without you” marginal player impact